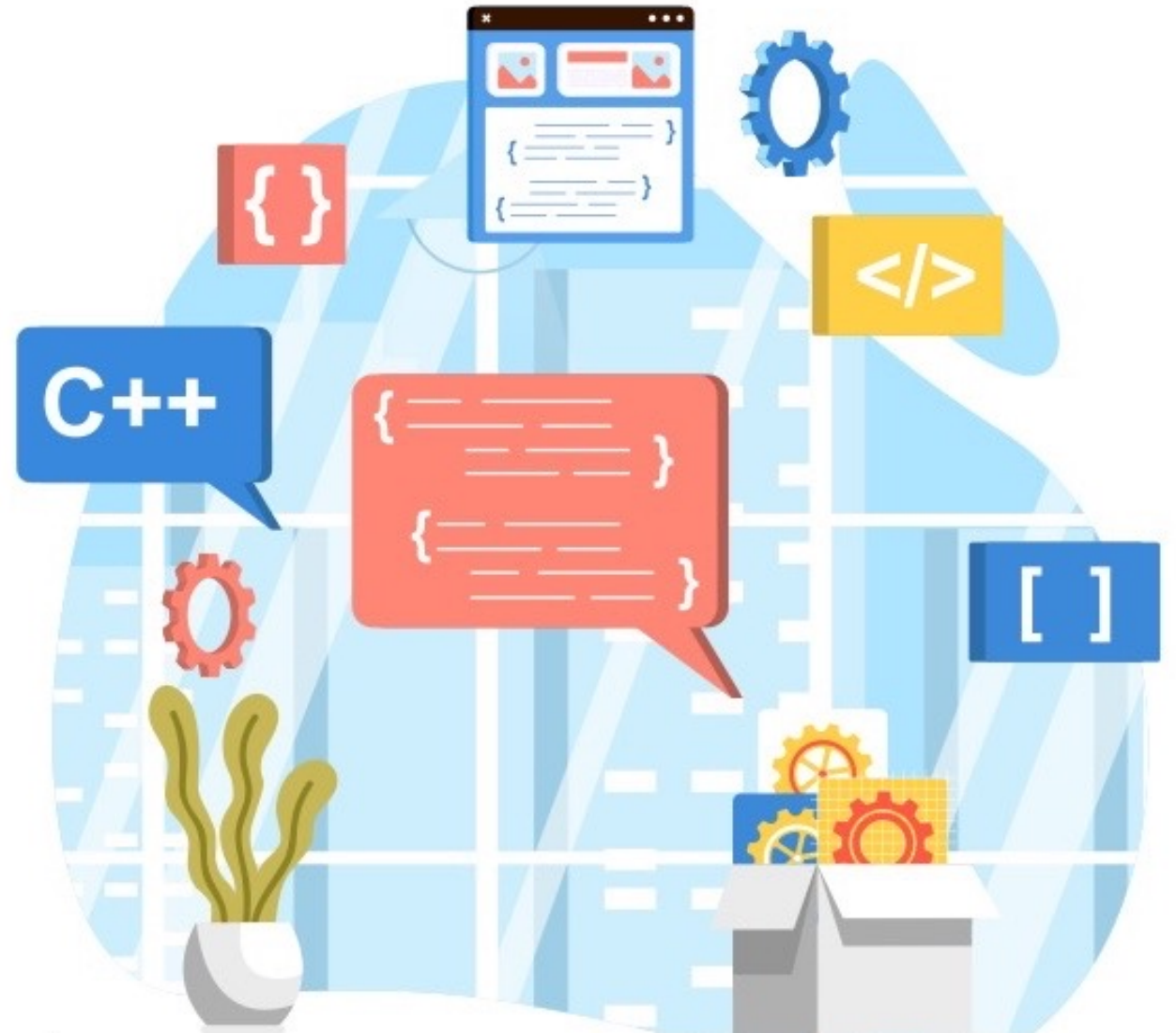


CÂY NHỊ PHÂN

VŨ NGỌC THANH SANG
KHOA CÔNG NGHỆ THÔNG TIN
ĐẠI HỌC SÀI GÒN

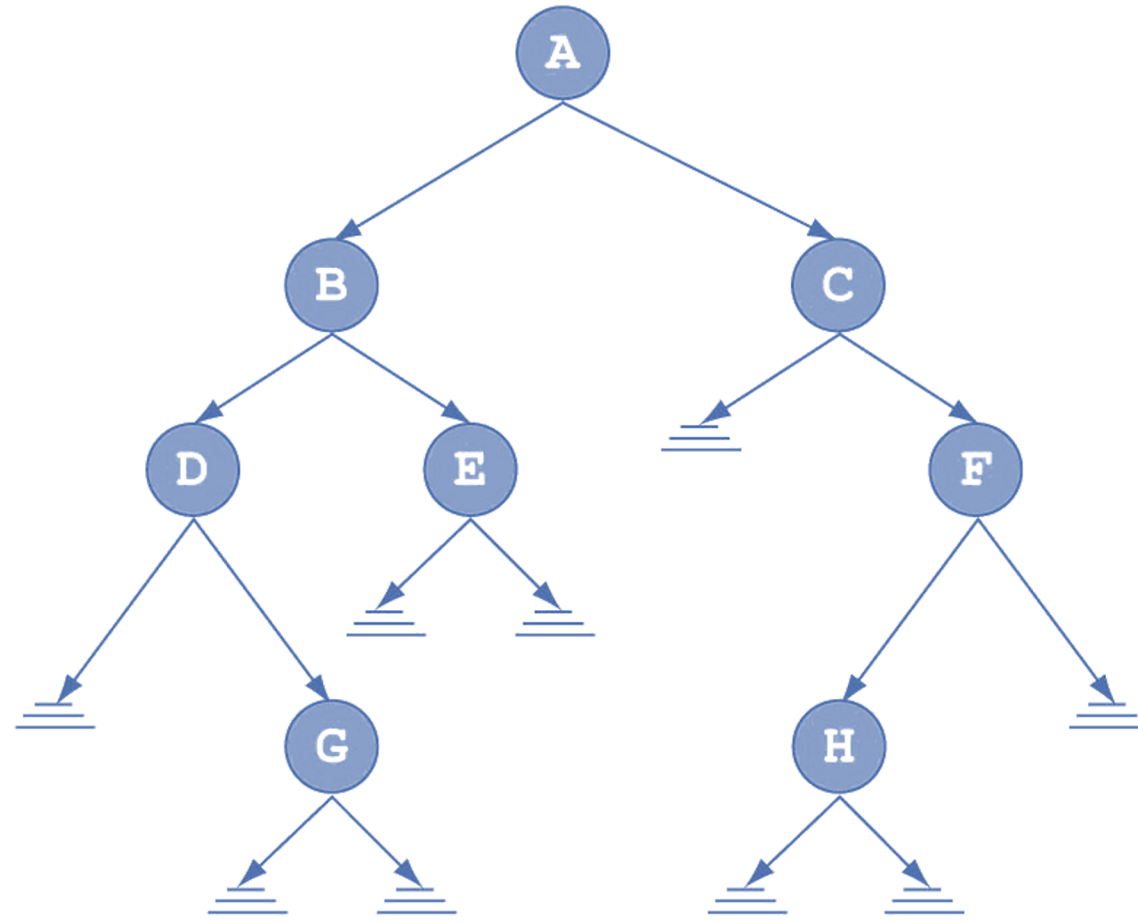


I - Introduction

- Cây nhị phân T nếu không rỗng sẽ có những đặc trưng sau:
 1. Có nút gốc
 2. Có 2 tập hợp nút:
 - Tập hợp nút con bên trái LT
 - Tập hợp nút con bên phải RT
 3. LT và RT đều là cây nhị phân

```
struct Node
{
    int info;
    struct Node *lLink;
    struct Node *rLink;
};
```

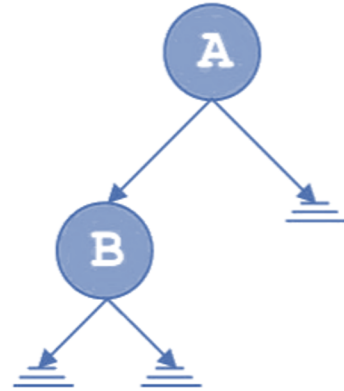
I - Introduction



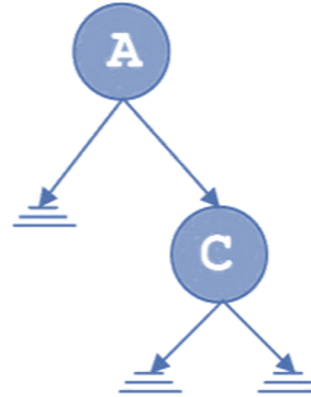
I - Introduction



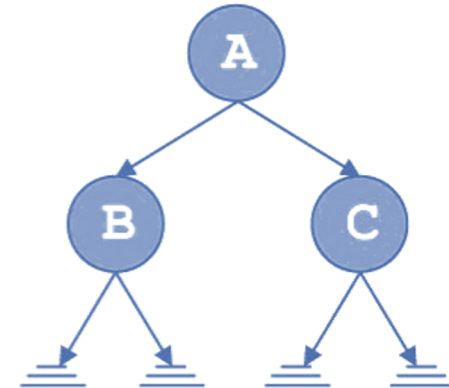
(a) Binary tree with one node



(b) Binary tree with two nodes

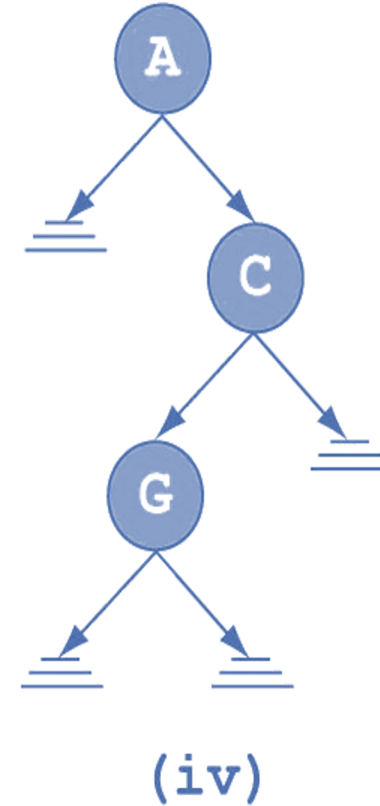
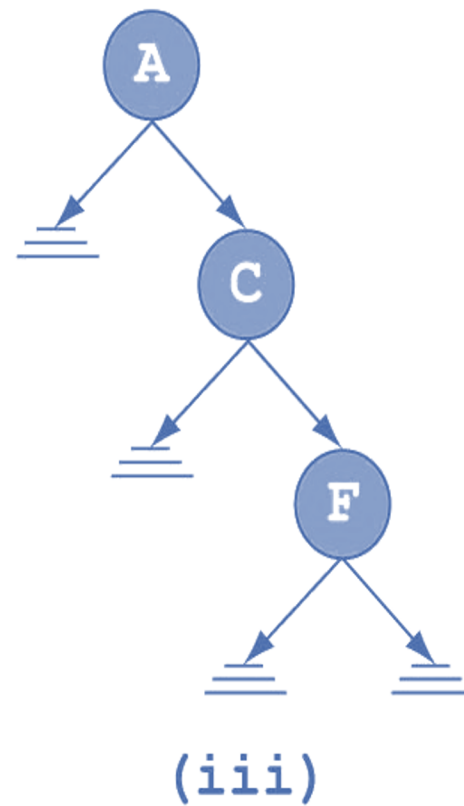
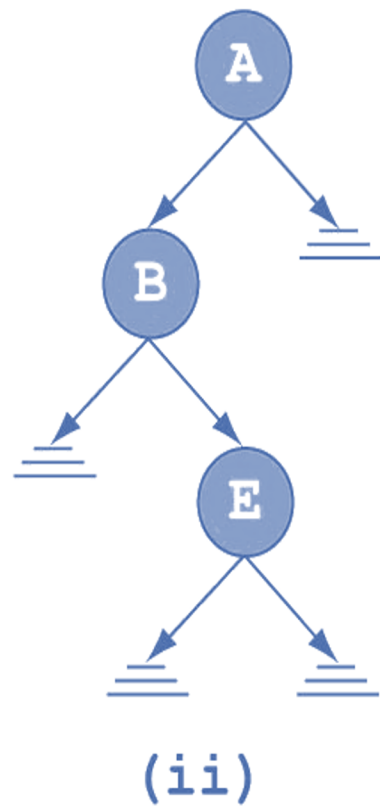
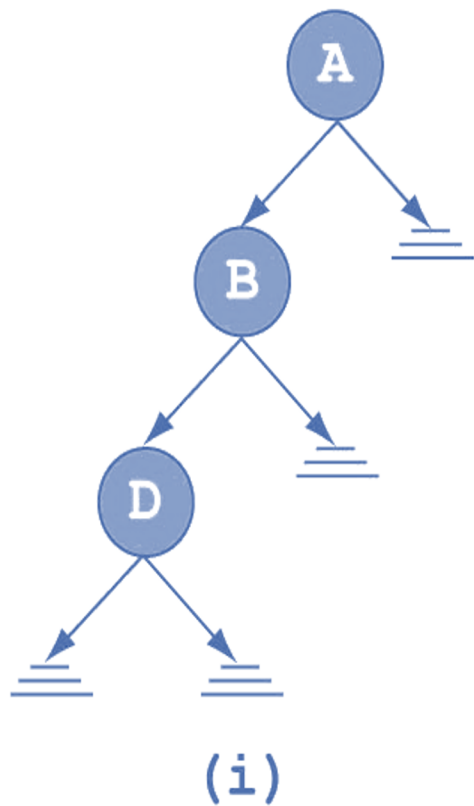


(c) Binary tree with two nodes



(d) Binary tree with three nodes

I - Introduction



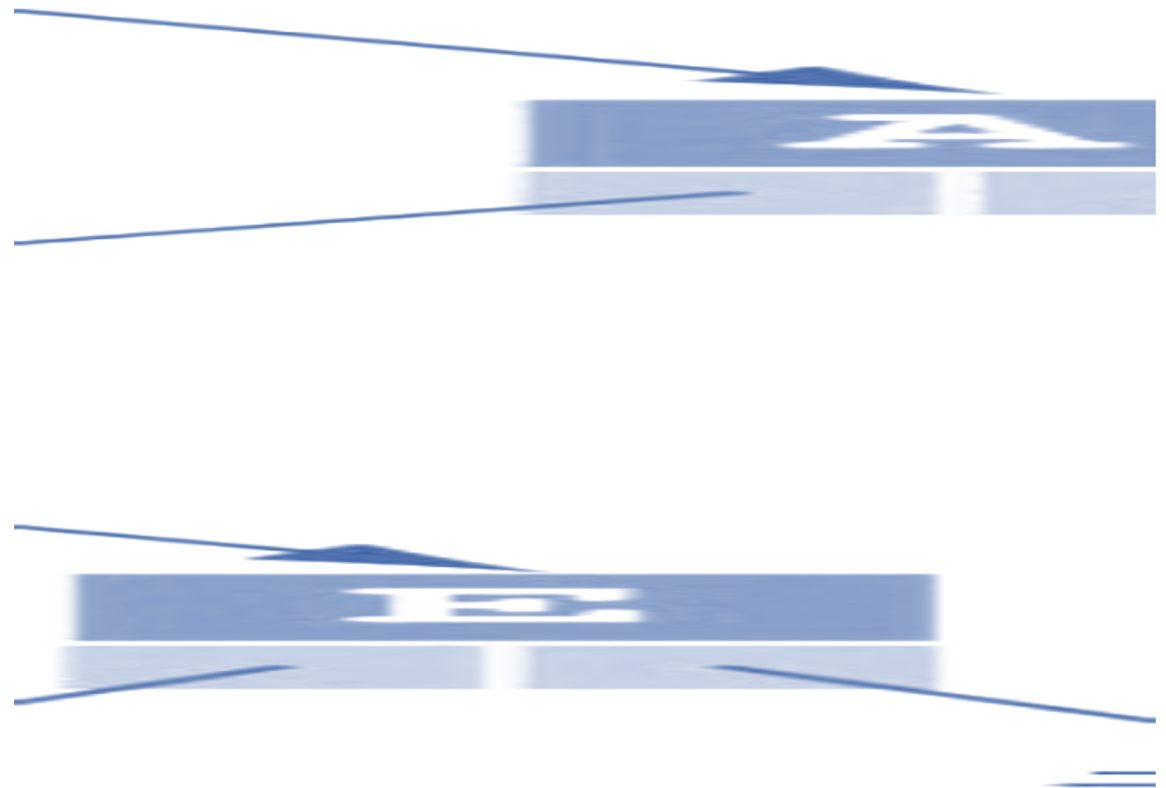
Các cây nhị phân có 3 nút

II - Binary trees

- Mỗi nút
 - có nhiều nhất là 2 nút con
 - chứa thông tin/giá trị của nó
 - chứa con trỏ trỏ tới tập hợp cây con bên trái (lLink) và tập hợp cây con bên phải (rLink)

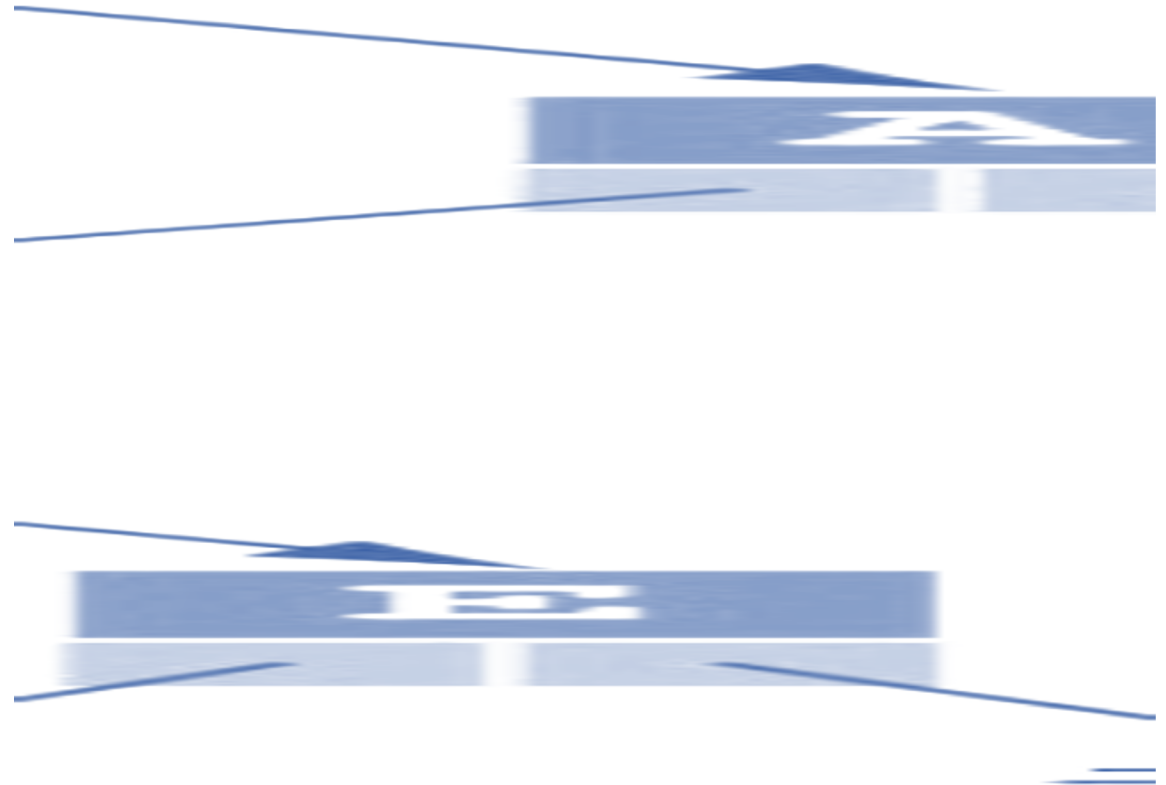
II - Binary trees

- **Nút lá (leaf node):**
→ nút không có nút con
- **Nút cha (parent node):**
→ nút nằm trên nhánh trái hoặc nhánh phải của một nút khác
- Có một **đường đi (path)** duy nhất từ nút gốc tới một nút lá bất kì
- **Độ dài đường đi (path length):**
→ số lần rẽ nhánh trên đường đi



II - Binary trees

- **Mức (level)** của một nút:
 - số lần rẽ nhánh trên đường đi từ nút gốc tới nó
 - nút gốc có mức 0
- **Chiều cao (height)** của cây:
 - là số nút nằm trên đường đi dài nhất từ nút gốc tới nút lá.
- **Kích thước (size)** của cây:
 - là tổng số nút của cây



II - Binary trees - Copy tree

- Cây nhị phân có cấu trúc dữ liệu động
 - Dữ liệu được cấp phát và thôi cấp phát trong thời gian thực thi
- Bản sao nông (shallow copy):
 - Trùng lặp ít nhất có thể
 - Sử dụng con trỏ để trỏ tới dữ liệu của cây.
- Bản sao sâu (deep copy)
 - Trùng lặp mọi thứ
 - Tạo từng nút và sao chép toàn bộ dữ liệu và con trỏ của cây

II - Binary trees - Traversal

- Chèn nút, xóa nút, tìm kiếm nút yêu cầu việc duyệt cây
 - Phải bắt đầu từ nút gốc
- Lựa chọn tại mỗi nút
 - Duyệt tiền tự (preoder traversal, node-left-right, NLR): duyệt nút gốc, duyệt tiền tự cây con trái, duyệt tiền tự cây con phải.
 - Duyệt trung tự (inorder traversal, left-node-right, LNR): duyệt trung tự cây con trái, duyệt nút gốc, duyệt trung tự cây con phải.
 - Duyệt hậu tự (postorder traversal, left-right-node, LRN): duyệt hậu tự cây con trái, duyệt hậu tự cây con phải, duyệt nút gốc.

<https://visualgo.net/en/bst>

II - Binary trees - Typical Operations

1. Xác định xem cây nhị phân có rỗng hay không
2. Tìm kiếm một phần tử bất kỳ trong cây nhị phân
3. Thêm một phần tử vào cây nhị phân
4. Xóa một phần tử khỏi cây nhị phân
5. Tìm độ cao của cây nhị phân
6. Tìm kích thước của cây nhị phân
7. Tìm số nhánh (lá) của cây nhị phân
8. Duyệt cây nhị phân
9. Sao chép cây nhị phân

II - Binary trees - Typical Operations

- Xác định độ cao của cây

```
int height(nodeType<int> *p) {  
    if (p == nullptr)  
        return 0;  
    else  
        return 1 + max(height(p->lLink), height(p->rLink));  
}
```

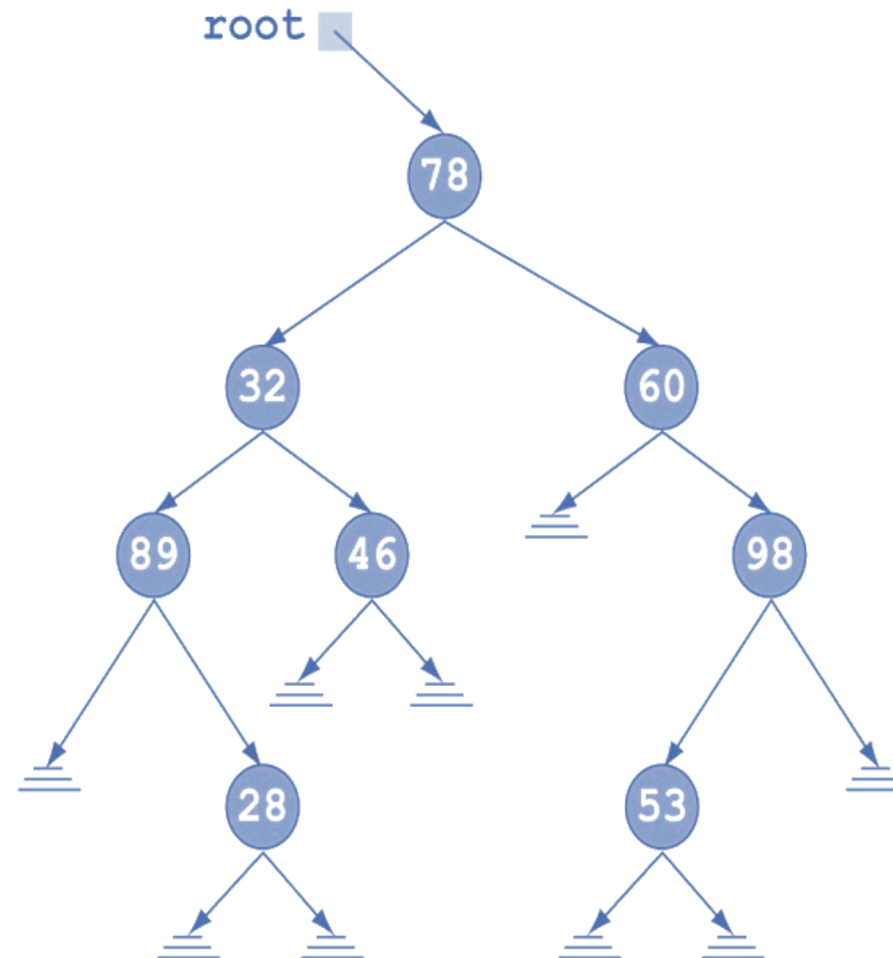

II - Binary trees - Typical Operations

- Sao chép cây

```
void copyTree(nodeType<int>* &copiedTreeRoot,  
              nodeType<int>* otherTreeRoot)  
{  
    if (otherTreeRoot == nullptr)  
        copiedTreeRoot = nullptr;  
    else  
    {  
        copiedTreeRoot = new nodeType<int>;  
        copiedTreeRoot->info = otherTreeRoot->info;  
        copyTree(copiedTreeRoot->lLink, otherTreeRoot->lLink);  
        copyTree(copiedTreeRoot->rLink, otherTreeRoot->rLink);  
    }  
} //endcopyTree
```

III - Binary search trees

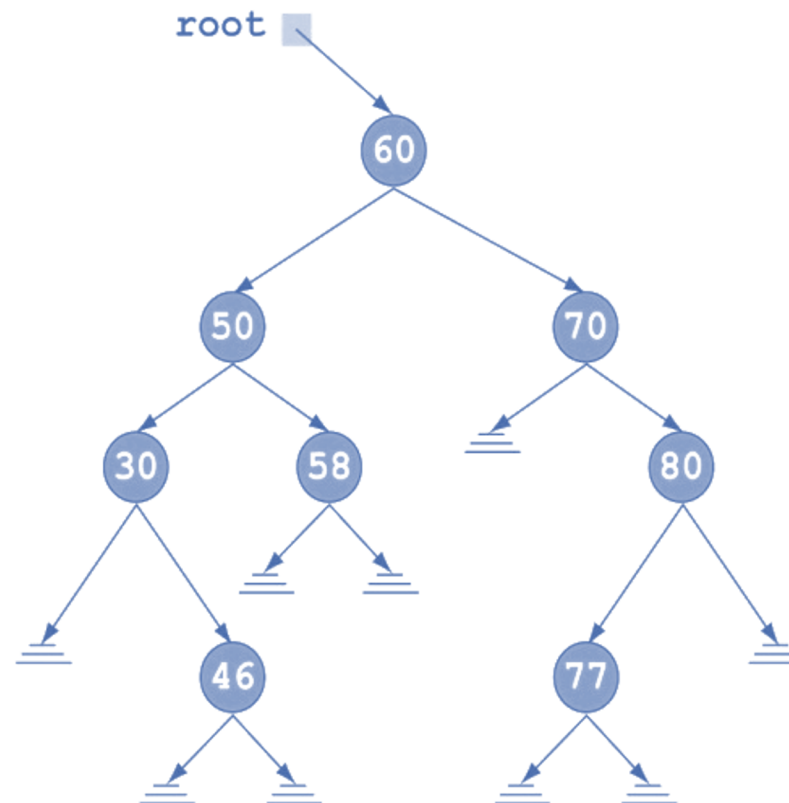
Xác định phần tử 53 có trong cây nhị phân sau hay không → chậm



III - Binary search trees

Trong cây tìm kiếm nhị phân, dữ liệu tại mỗi nút:

- Lớn hơn dữ liệu của nút con bên trái
- Nhỏ hơn dữ liệu của nút con bên phải



III - Binary search trees

Cây tìm kiếm nhị phân nếu không rỗng sẽ có các đặc trưng sau:

- Có nút gốc
- Có hai tập hợp nút: tập nút con bên trái LT và tập nút con bên phải RT
- Giá trị của nút gốc lớn hơn giá trị của tất cả các nút con của nhánh bên trái, và nhỏ hơn giá trị của tất cả nút con của nhánh bên phải.
- LT và RT là cây tìm kiếm nhị phân

III - Binary search trees - Typical Operations

1. Xác định xem cây có rỗng hay không
2. Tìm kiếm một phần tử cụ thể
3. Thêm hoặc xóa phần tử khỏi một cây
4. Xác định độ cao của cây
5. Tìm số kích thước và số nút lá của cây
6. Duyệt cây
7. Sao chép cây

III - Binary search trees - Search

Bắt đầu tìm kiếm tại nút gốc

Nếu không tìm thấy

 Tìm kiếm ở nhánh bên trái

Nếu không tìm thấy ở nhánh bên trái

 Tìm kiếm ở nhánh bên phải

Nếu không tìm kiếm ở nhánh bên phải

 Phần tử cần tìm không có trong cây

III - Binary search trees - Insert

- Sau khi thêm phần tử vào cây, kết quả sẽ thu được một cây nhị phân mới
- Tìm vị trí cho phần tử mới

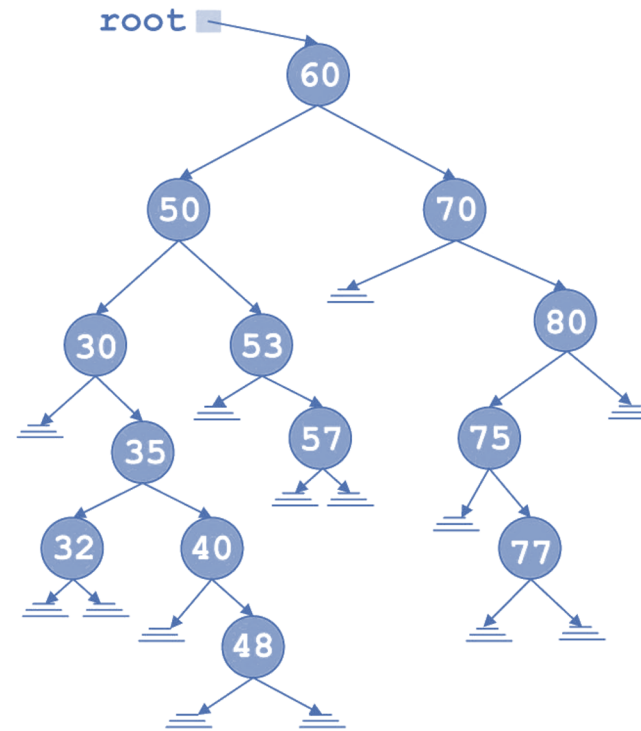
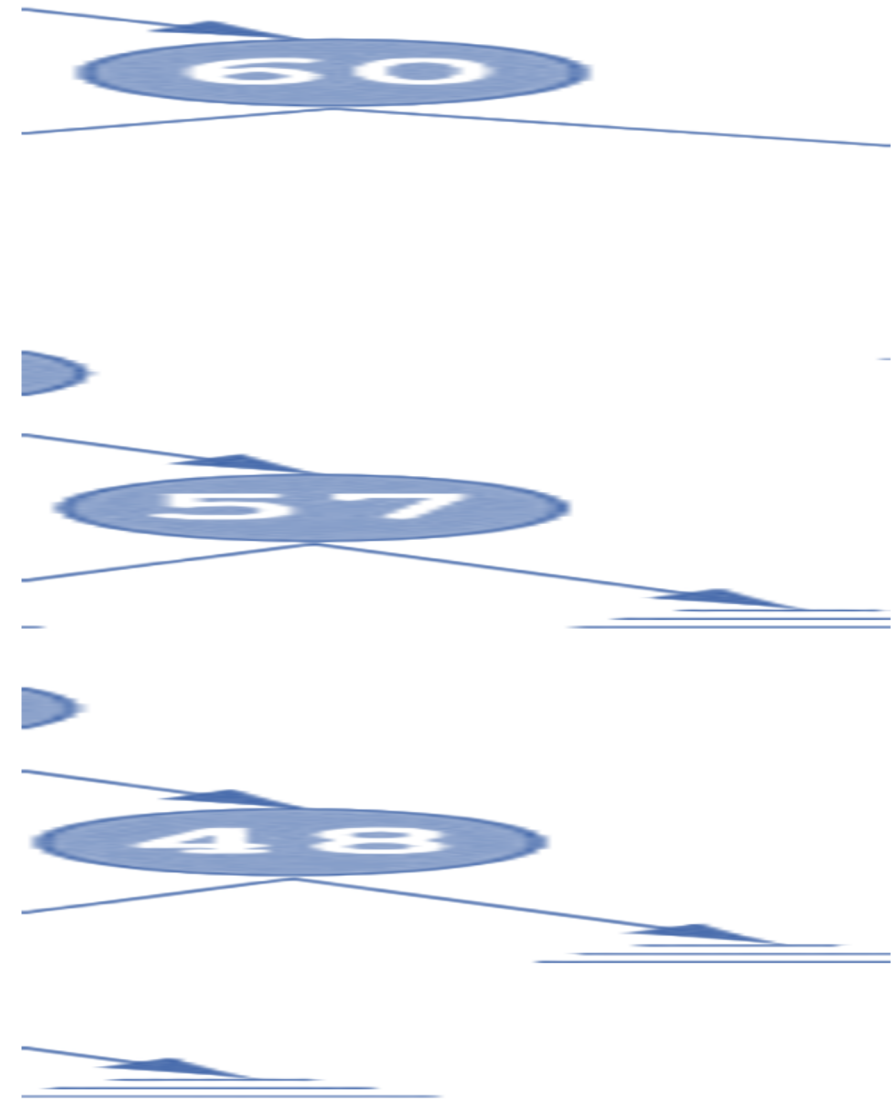
Sử dụng hai con trỏ trỏ tới địa chỉ của nút hiện tại và nút cha của nó

III - Binary search trees - Delete

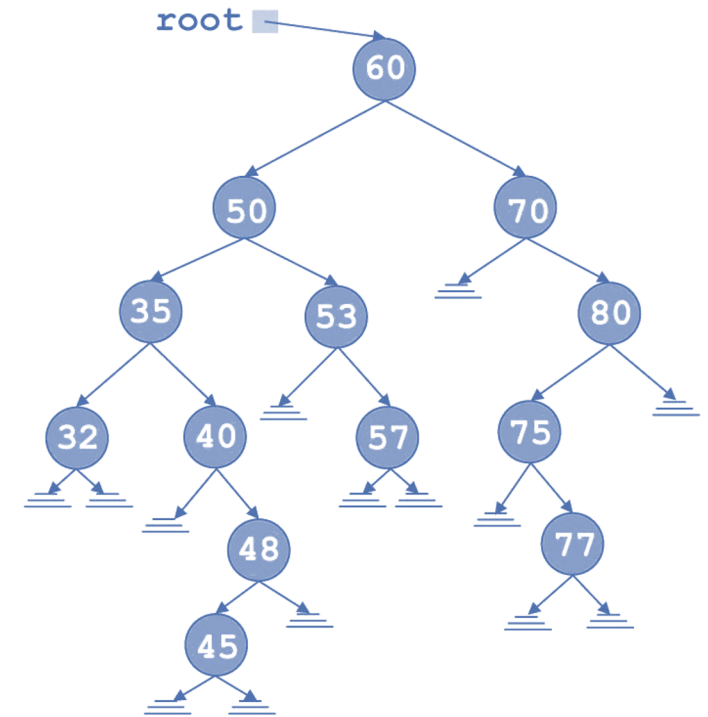
Để xóa một nút khỏi cây, xem xét 4 trường hợp

- Nút bị xóa là nút lá
- Nút bị xóa không có nhánh bên trái
- Nút bị xóa không có nhánh bên phải
- Nút bị xóa có cả nhánh trái và nhánh phải (không rỗng)

III - Binary search trees - Delete



(a) Delete 45



(b) Delete 30

III - Binary search trees - Analysis

Để xóa một nút khỏi cây, xem xét 4 trường hợp

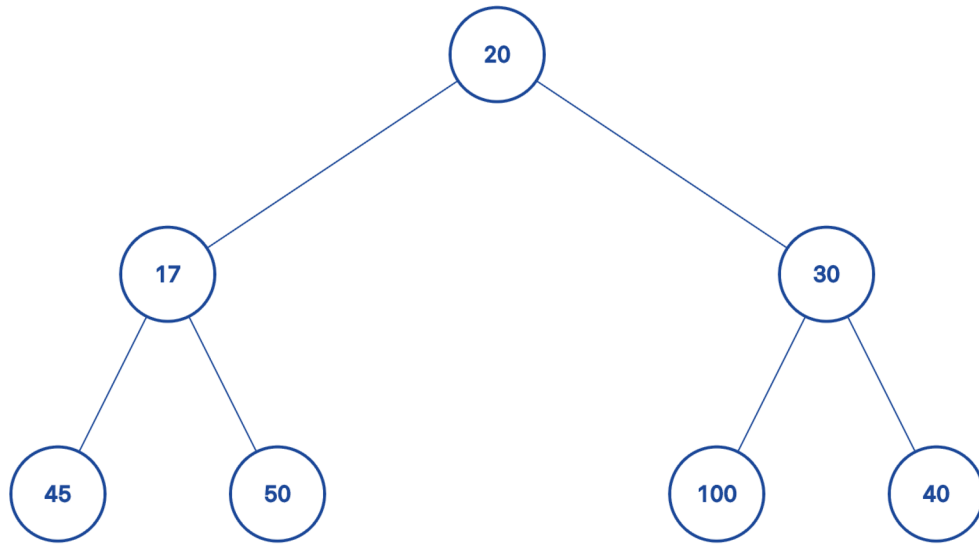
- Nút bị xóa là nút lá
- Nút bị xóa không có nhánh bên trái
- Nút bị xóa không có nhánh bên phải
- Nút bị xóa có cả nhánh trái và nhánh phải (không rỗng)

1. Viết hàm tìm kiếm phần tử trong cây tìm kiếm nhị phân
2. Viết hàm chèn phần tử vào cây tìm kiếm nhị phân
3. Viết hàm xóa phần tử khỏi cây tìm kiếm nhị phân
4. Viết hàm không sửa dụng đệ quy để duyệt cây tiền tự, trung tự và hậu tự

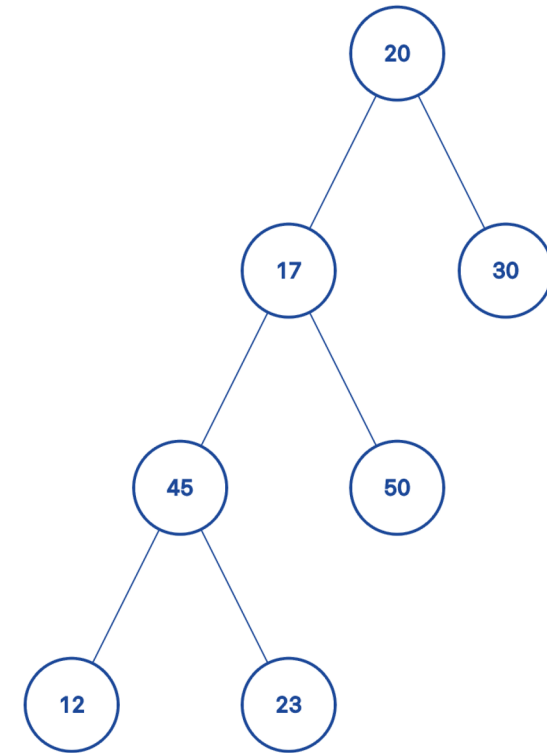
III - Other Binary Trees - Full Binary Tree

- Một cây được xem là cây nhị phân đầy đủ khi mọi nút của nó có 0 hoặc 2 nút con.
- Trong một cây nhị phân đầy đủ, số lượng các nút lá (leaf nodes) bằng với số lượng các nút bên trong (internal node) + 1.

III - Other Binary Trees - Full Binary Tree



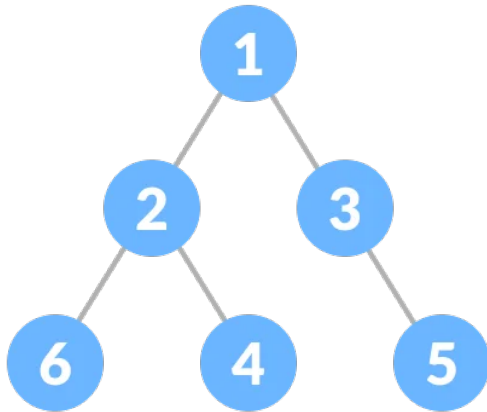
- Nút gốc: 20
Nút bên trong: 17, 30
- Nút lá: 45, 50, 100, 40



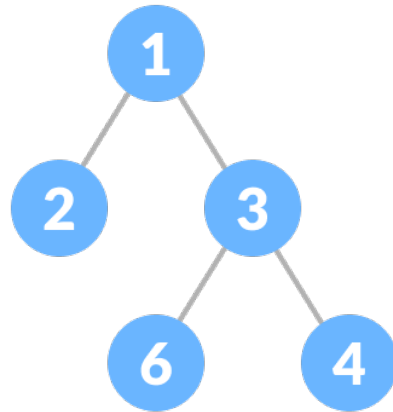
- Nút gốc: 20
Nút bên trong: 17, 45
- Nút lá: 12, 23, 50, 30

III - Other Binary Trees - Complete Binary Tree

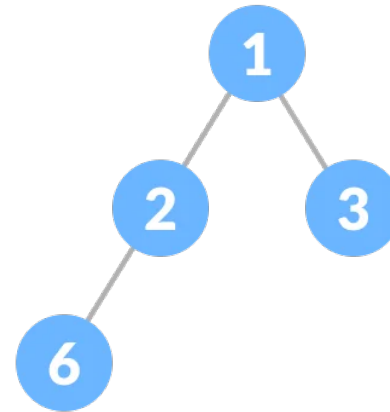
- Một cây được xem là cây nhị phân hoàn chỉnh nếu tất cả các mức (level) của nó ngoại trừ mức cuối cùng đều được lấp đầy bởi các nút.
- Mức cuối cùng được thêm vào từ bên trái



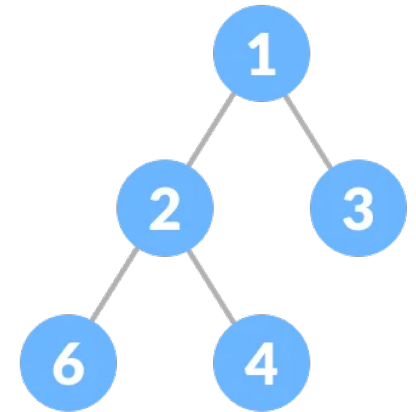
✗ Full Binary Tree
✗ Complete Binary Tree



✓ Full Binary Tree
✗ Complete Binary Tree



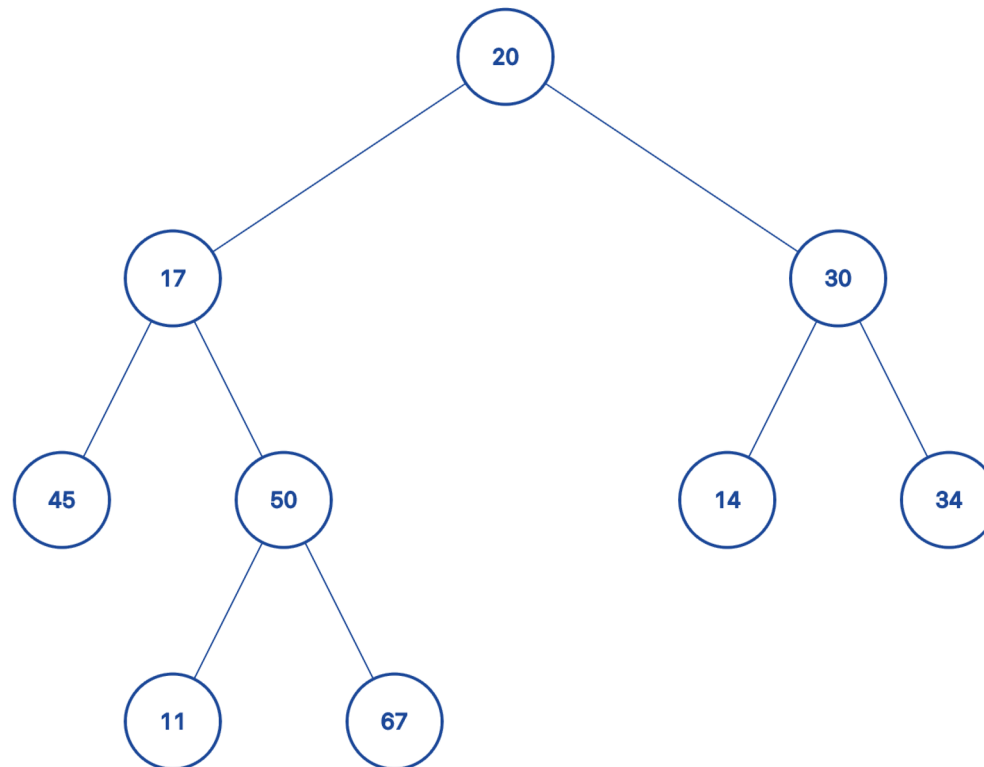
✗ Full Binary Tree
✓ Complete Binary Tree



✓ Full Binary Tree
✓ Complete Binary Tree

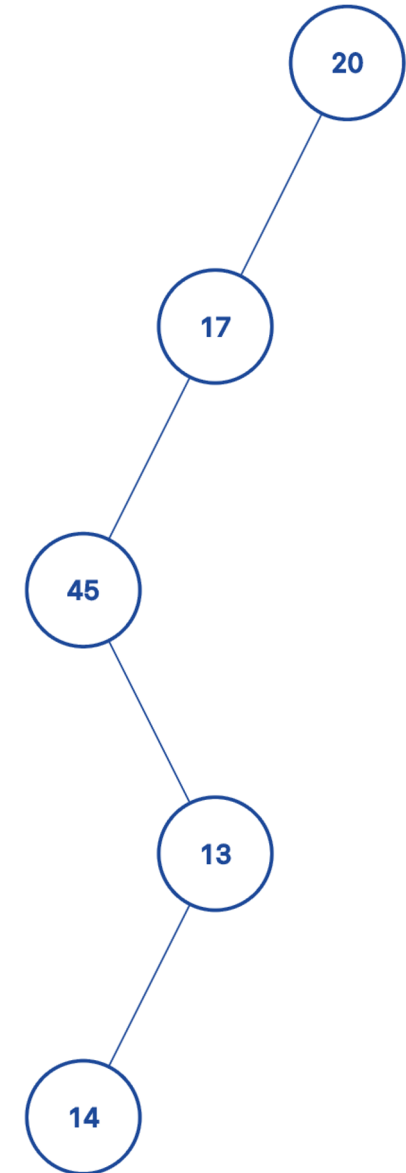
III - Other Binary Trees - Perfect Binary Tree

- Một cây được xem là cây nhị phân hoàn hảo nếu tất cả các nút bên trong của nó đều có đủ hai nút con và tất cả các nút lá đều ở cùng mức (level).



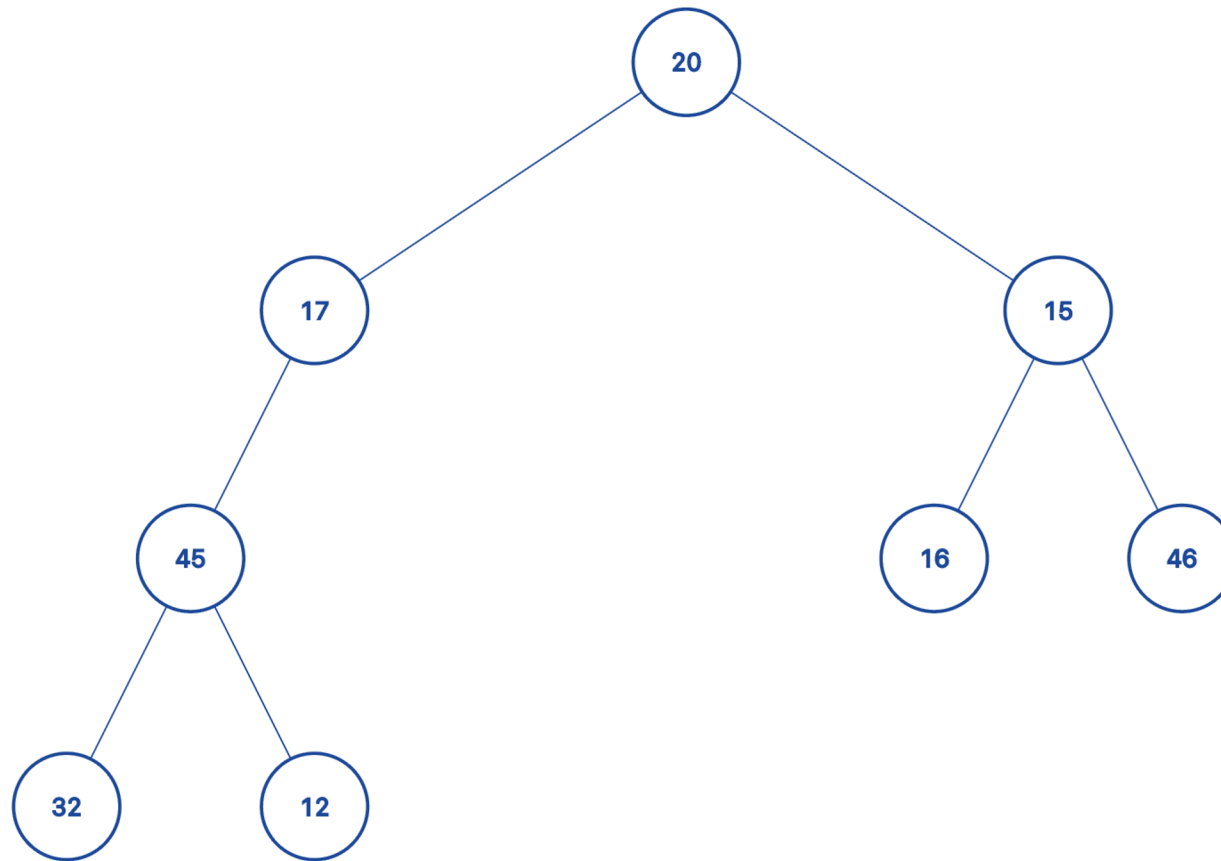
III - Other Binary Trees - Degenerate Tree

- Một cây được xem là cây nhị phân thoái hóa là tại mỗi nút bên trong đều chỉ có một nút con.
- Có hiệu suất hoạt động tương tự với DSLK



III - Other Binary Trees - Balanced Binary Tree

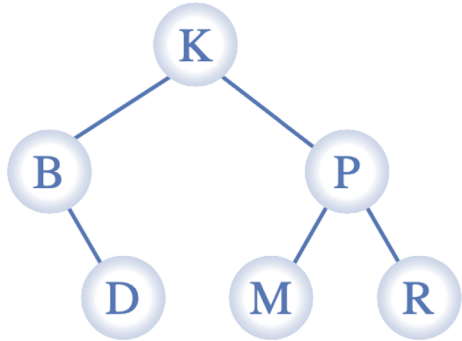
- Một cây được xem là cây nhị phân cân bằng nếu sự khác nhau giữa chiều cao của nhánh con bên trái và nhánh con bên phải tối đa là 1.



IV – Balancing a Tree

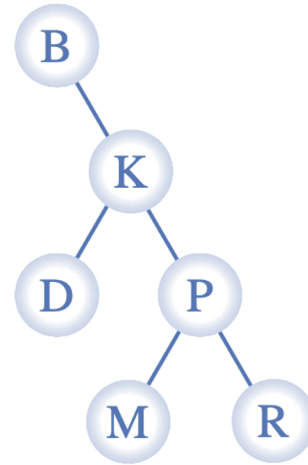
- Cây tìm kiếm nhị phân
 1. Biểu diễn dữ liệu dạng phân cấp theo một thứ tự cụ thể
 2. Tìm kiếm trên cây tìm kiếm nhị phân hiệu quả hơn tìm kiếm trên mảng
- Tuy nhiên, hiệu quả tìm kiếm phụ thuộc vào cấu trúc của cây. Tìm kiếm dữ liệu trên cây bị lệch không hiệu quả hơn tìm kiếm tuần tự trên mảng.
- Cấu trúc của cây tìm kiếm nhị phân sẽ được quyết định bởi thứ tự thêm vào của các nút.

IV - Balancing a Tree



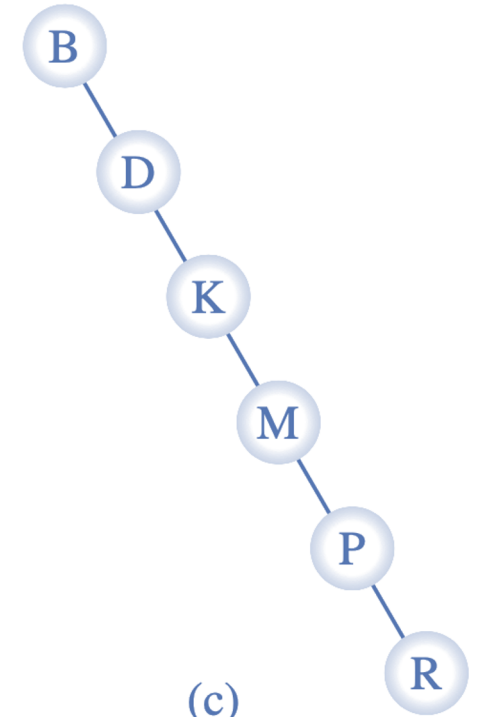
(a)

A fairly well-balance tree



(b)

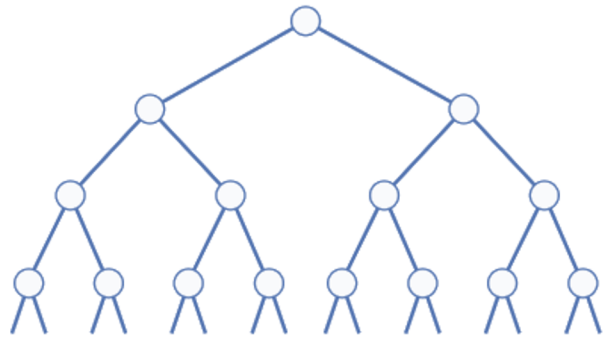
A right unbalance tree



(c)

A right skewed tree

IV - Balancing a Tree



Height	Nodes at One Level	Nodes at All Levels
1	$2^0 = 1$	$1 = 2^1 - 1$
2	$2^1 = 2$	$3 = 2^2 - 1$
3	$2^2 = 4$	$7 = 2^3 - 1$
4	$2^3 = 8$	$15 = 2^4 - 1$
⋮		
11	$2^{10} = 1,024$	$2,047 = 2^{11} - 1$
⋮		
14	$2^{13} = 8,192$	$16,383 = 2^{14} - 1$
⋮		
h	2^{h-1}	$n = 2^h - 1$
⋮		

- Chiều cao của cây tìm kiếm nhị phân cân bằng là $\lceil \log(n + 1) \rceil$

IV – Balancing a Tree

- Cây tìm kiếm nhị phân cân bằng được dùng để chứa 10,000 phần tử sẽ có chiều cao $\lceil \log(10001) \rceil = \lceil 13.288 \rceil = 14$
- Từ nút gốc, chúng ta cần tối đa 14 lần tìm kiếm (lần lặp) để tìm kiếm một phần tử cụ thể.
- Nếu tổ chức dữ liệu dưới dạng DSLK, chúng ta cần 10,000 lần lặp cho trường hợp xấu nhất.
- Vì vậy cấu tạo cây tìm kiếm nhị phân cân bằng là giải pháp hiệu quả cho bài toán tìm kiếm.

IV – Balancing a Tree

Xây dựng cây tìm kiếm nhị phân cân bằng

1. Sắp xếp mảng 1 chiều theo chiều tăng dần.
2. Phần tử ở giữa mảng sẽ trở thành nút gốc
 - 2.1. Phần tử ở giữa mảng bên trái sẽ trở thành nút đầu tiên của nhánh bên trái.
 - 2.2. Phần tử ở giữa mảng bên phải sẽ trở thành nút đầu tiên của nhánh bên phải.
3. Lặp lại quá trình trên để xây dựng cây theo từng mức

IV - Balancing a Tree

Stream of data: 5 1 9 8 7 0 2 3 4 6

Array of sorted data: 0 1 2 3 4 5 6 7 8 9

(a) 0 1 2 3 4 5 6 7 8 9

4

(b) 0 1 2 3 4 5 6 7 8 9

4

1

(c) 0 1 2 3 4 5 6 7 8 9

4

1

0

2

(d) 0 1 2 3 4 5 6 7 8 9

4

1

7

0

2

5

8

3

6

9

IV - Balancing a Tree

```
void balanceBST(int data[], int first, int last) {  
    if (first <= last) {  
        int middle = (first + last)/2;  
  
        // chèn nút vào cây  
        insert(data[middle]);  
  
        // thực hiện đệ quy cho nhánh bên trái và bên phải  
        balance(data, first, middle-1);  
        balance(data, middle+1, last);  
    }  
}
```

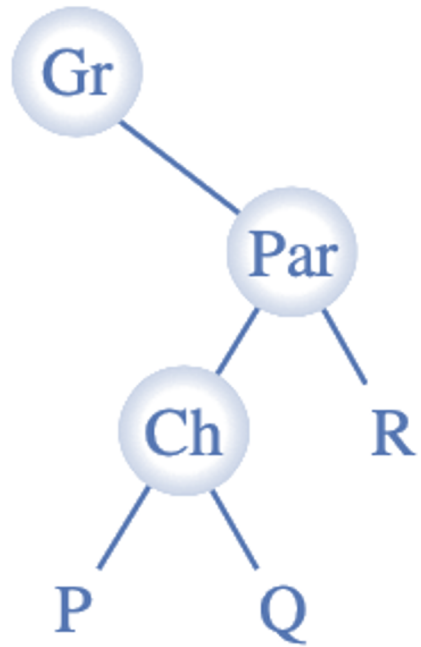

IV - Balancing a Tree

- Thuật toán này có hạn chế là toàn bộ dữ liệu phải được chứa và sắp xếp ở dạng mảng trước khi cây tìm kiếm nhị phân được tạo.
- Cây không cân bằng có thể được cân bằng bằng cách sử dụng duyệt trung tự và ghi dữ liệu đầu ra vào mảng

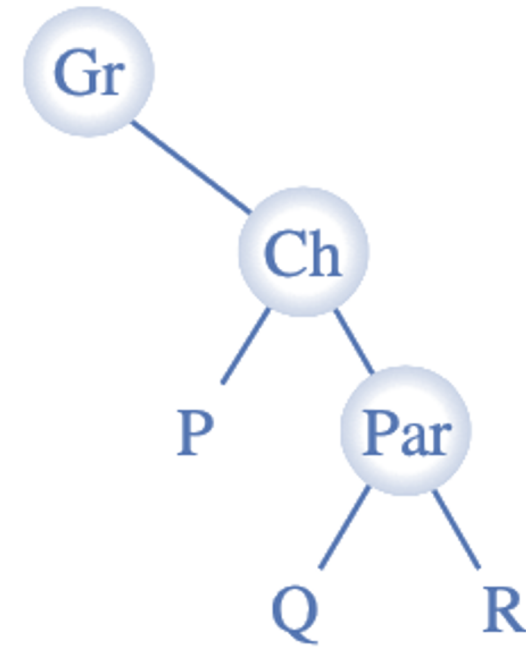
IV – Balancing a Tree - DSW Algorithm

- Thuật toán được phát triển bởi Colin Day và được điều chỉnh bởi Quentin F.Stout và Better L.Warren nên được viết tắt là DSW.
- Ý tưởng của thuật toán là sử dụng phép xoay.
 - Có 2 hướng xoay: xoay trái và xoay phải.
 - Sau khi thực hiện phép xoay, có thể đổi vai trò của nút cha và nút con.
- Tính chất của cây tìm kiếm nhị phân cần được bảo toàn sau khi thực hiện phép xoay.
- Thuật toán biến đổi cây tìm kiếm nhị phân bất kì thành cấu trúc gần giống với DSLK gọi là khung xương (backbone).

IV - Balancing a Tree - DSW Algorithm



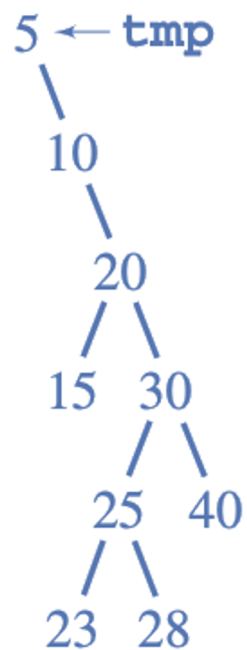
(a)



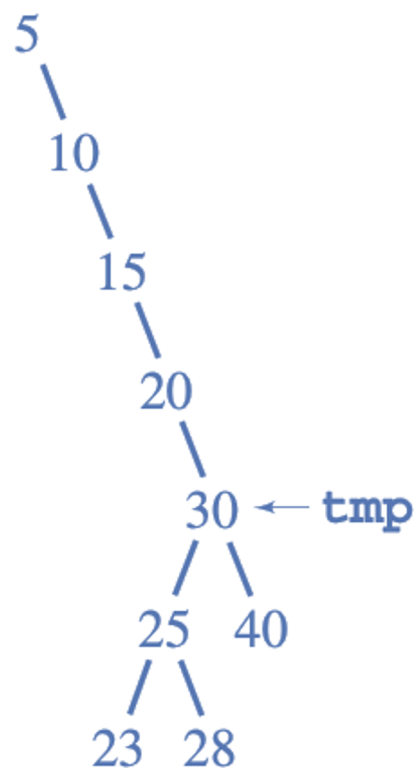
(b)

IV - Balancing a Tree - DSW Algorithm

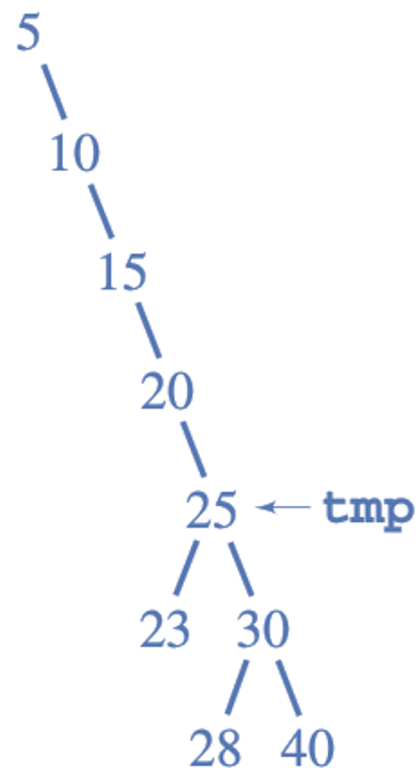
- Bước 1: Khởi tạo khung xương



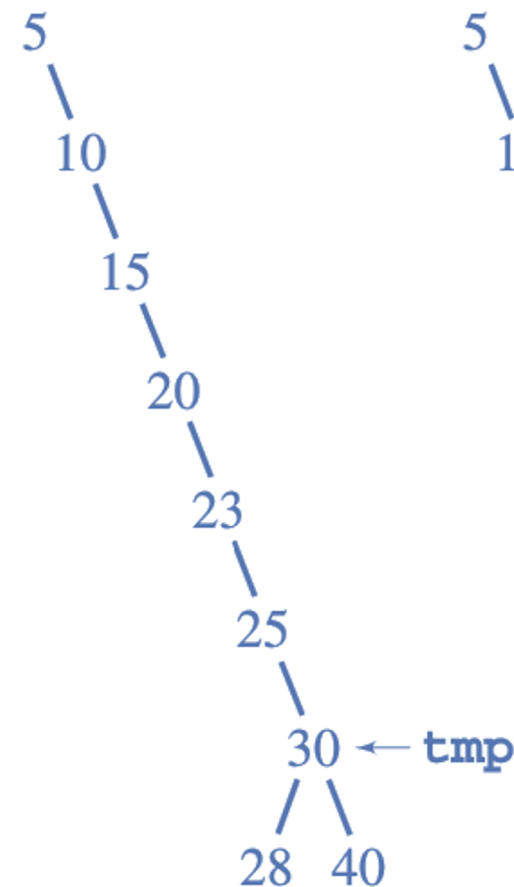
(a)



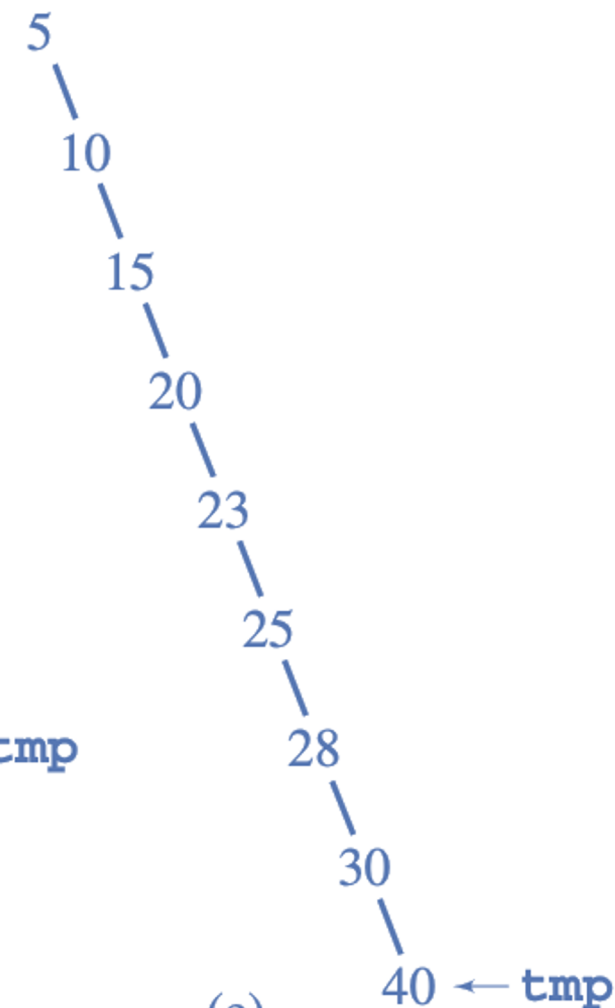
(b)



(c)



(d)



(e)

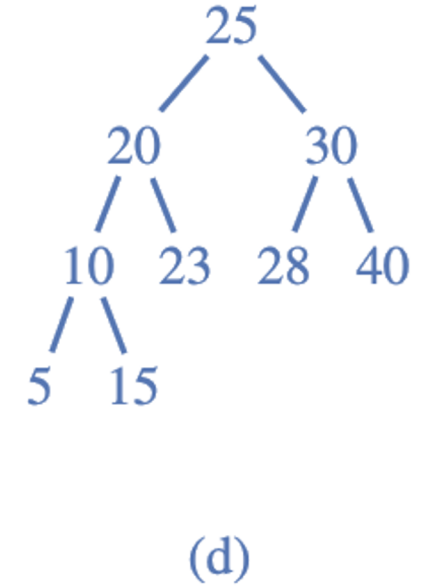
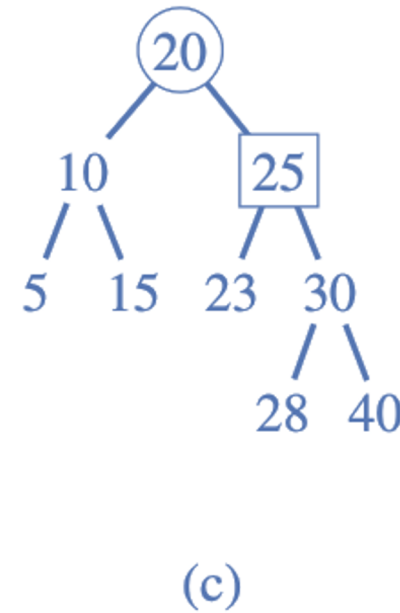
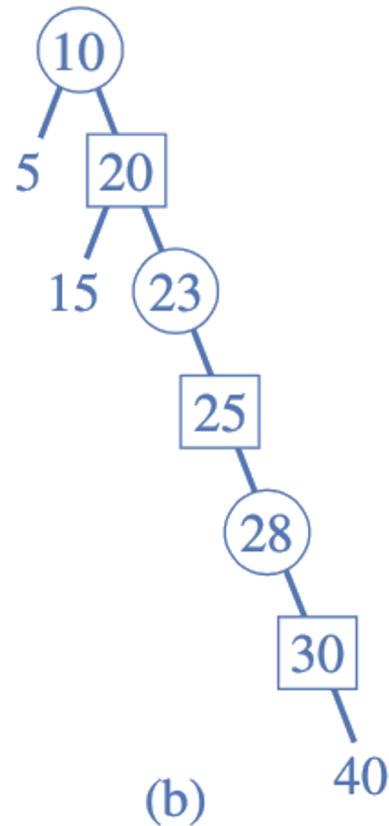
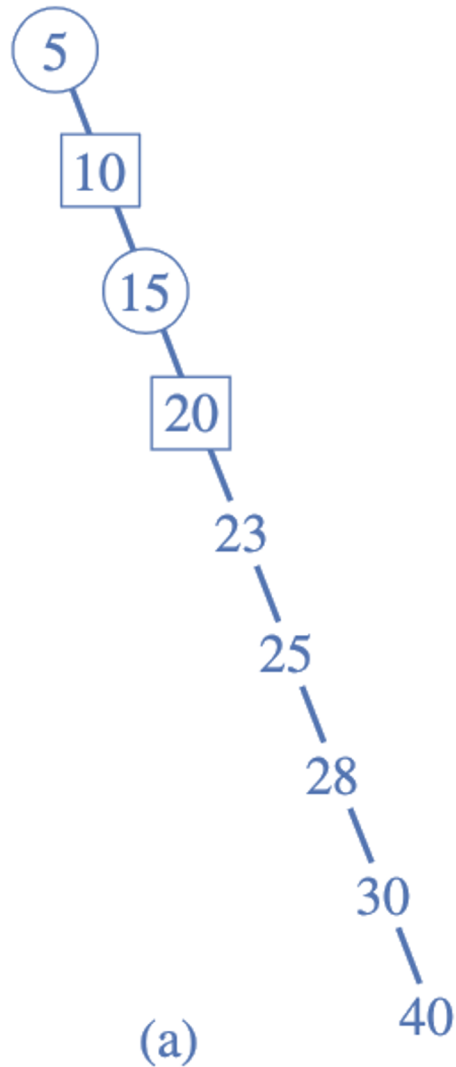
IV - Balancing a Tree - DSW Algorithm

- Bước 1: Khởi tạo khung xương

```
createBackbone(root)
  tmp = root;
  while (tmp != 0)
    if tmp has a left child
      // hence the left child becomes parent of tmp
      rotate this child about tmp;
      set tmp to the child that just became parent;
    else set tmp to its right child;
```

IV - Balancing a Tree - DSW Algorithm

- Bước 2: Biến đổi khung xương thành cây tìm kiếm nhị phân cân bằng



IV - Balancing a Tree - DSW Algorithm

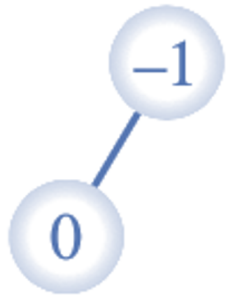
- Bước 2: Biến đổi khung xương thành cây tìm kiếm nhị phân cân bằng

```
createPerfectTree()  
    n = number of nodes;  
    m = 2^[lg(n+1)]-1;  
    make n-m rotations starting from the top of backbone;  
    while (m > 1)  
        m = m/2;  
        make m rotations starting from the top of backbone;
```

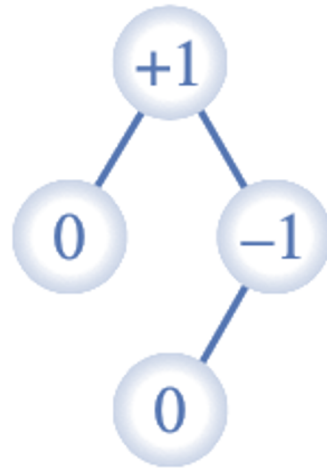
IV - Balancing a Tree - AVL Trees

- Thuật toán DSW có hạn chế là việc chèn nút hoặc xóa nút chỉ ảnh hưởng tới một phần của cây, mang tính cục bộ.
- Giải pháp cho vấn đề trên là sửa dụng cây AVL được phát triển bởi Adel'son-Vel'skii and Landis.
- Cây AVL là cây có sự khác biệt giữa nhánh trái và nhánh phải không quá 1 nút.
- Nhân tố cân bằng (balance factor) là sự khác biệt chiều cao giữa nhánh bên phải và nhánh bên trái. Nhân tố có các giá trị + 1; 0 và -1.

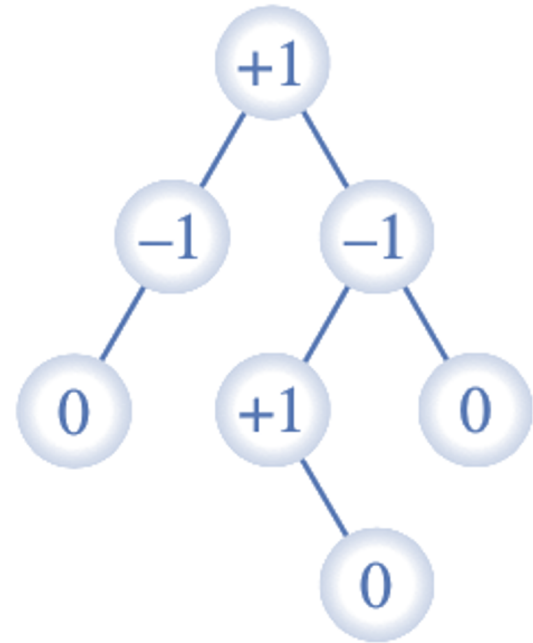
IV - Balancing a Tree - AVL Trees



(a)



(b)



(c)

IV - Balancing a Tree - AVL Trees - Node Insertion

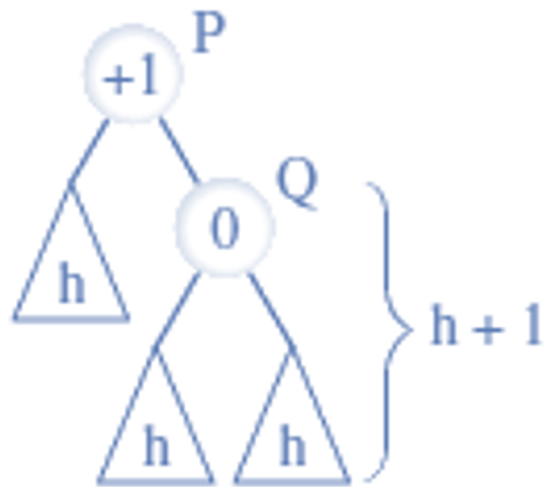
- Số lượng nút tối thiểu của cây AVL được xác định bởi:

$$AVL_h = AVL_{h-1} + AVL_{h-2} + 1$$

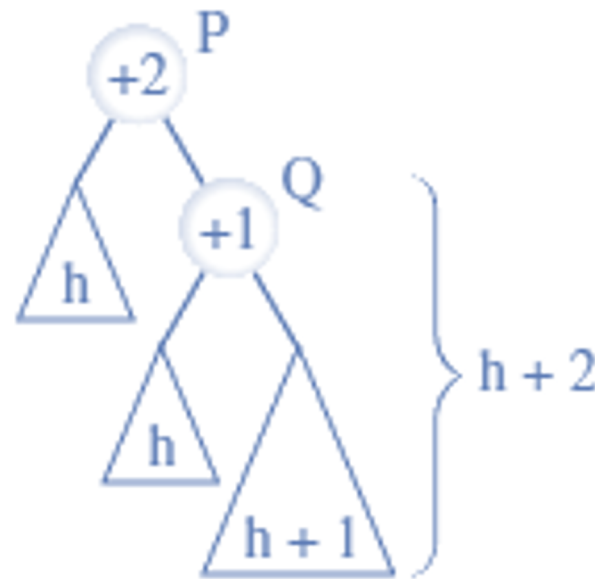
- Khi bất kì nút nào trong cây AVL có nhân tố cân bằng nhỏ hơn -1 hoặc lớn hơn 1, node đó cần phải được cân bằng.
- Có 4 trường hợp một cây có thể trở nên mất cân bằng
 - ✓ TH1: Khi một node được chèn vào cây con bên phải của nút con bên phải
 - ✓ TH2: Khi một node được chèn vào cây con bên trái của nút con bên phải
 - ✓ TH3: Khi một node được chèn vào cây con bên phải của nút con bên trái
 - ✓ TH4: Khi một node được chèn vào cây con bên trái của nút con bên trái

IV - Balancing a Tree - AVL Trees - Node Insertion

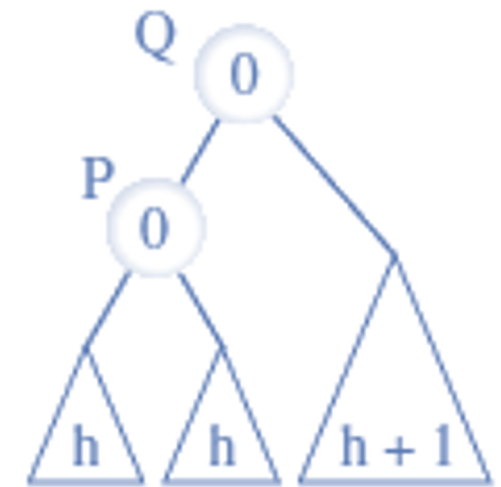
- TH1: Khi một nút được chèn vào cây con bên phải của nút con bên phải
- Sau khi một nút mới được chèn vào đâu đó trong cây con bên phải của Q để làm mất cân bằng cây, Q xoay xung quanh là nút cha P để cân bằng lại cây



(a)



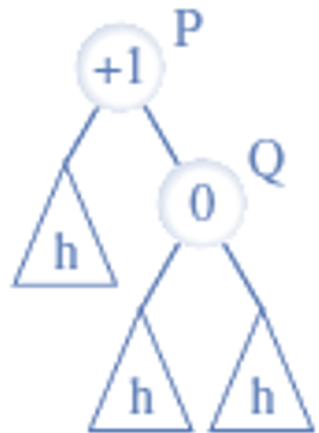
(b)



(c)

IV - Balancing a Tree - AVL Trees - Node Insertion

- TH2: Khi một nút được chèn vào cây con bên trái của nút con bên phải
- Sự mất cân bằng được giải quyết bằng một phép quay kép: đầu tiên R xung quanh Q (d), sau đó R xung quanh P (e)



(a)



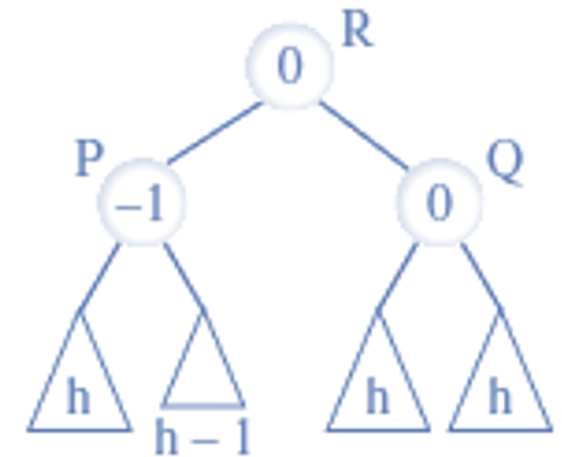
(b)



(c)



(d)

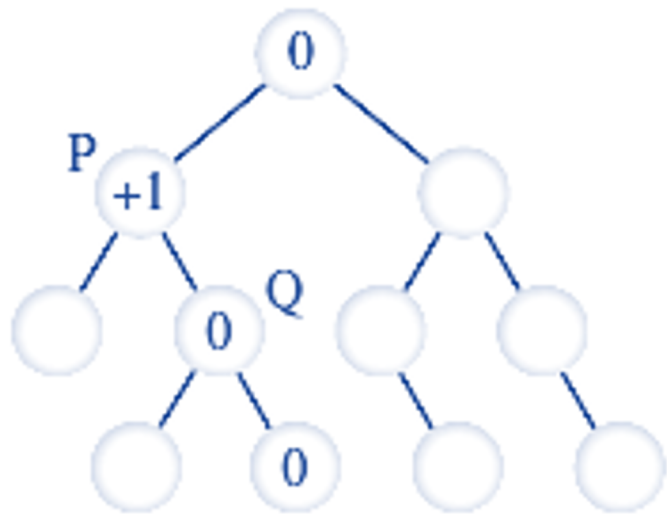


(e)

IV - Balancing a Tree - AVL Trees - Node Insertion

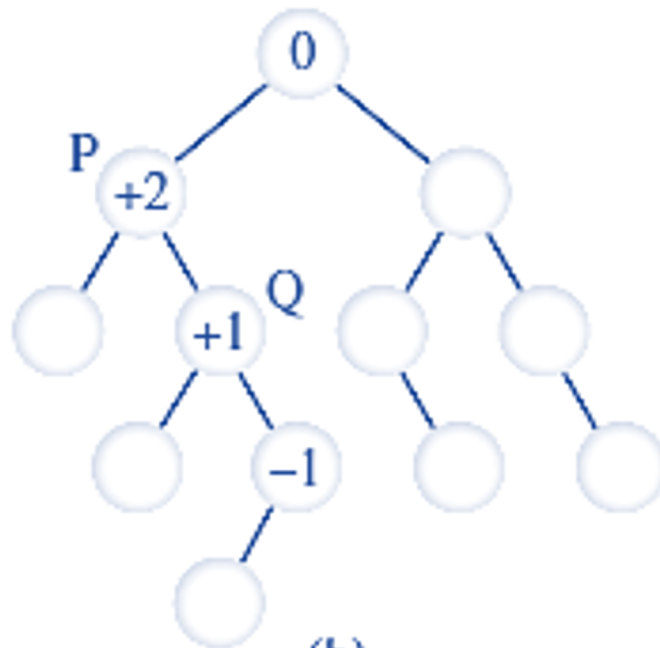
- Ví dụ trên xem P như một cây độc lập, tuy nhiên P vẫn có thể là một nhánh của cây lớn hơn.
- Bài toán đổi thành tìm node P sau khi chèn nút làm cho cây mất cân bằng sau khi chèn.
- Từ nút mới được chèn, cập nhật nhân tố cân bằng cho tới khi nó bằng ± 2 . Nút này được xem như node P hay là nút gốc của nhánh cần được cân bằng.

IV - Balancing a Tree - AVL Trees - Node Insertion



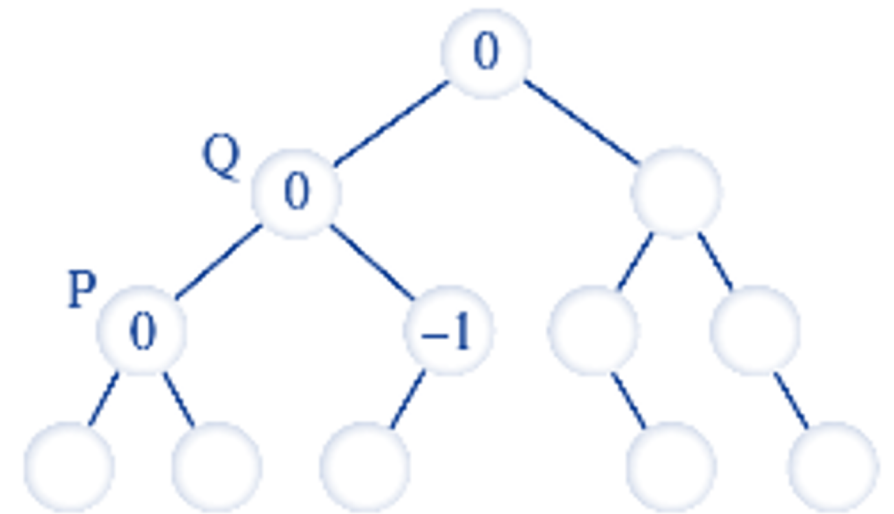
(a)

a) Cây AVL



(b)

b) Chèn nút mới



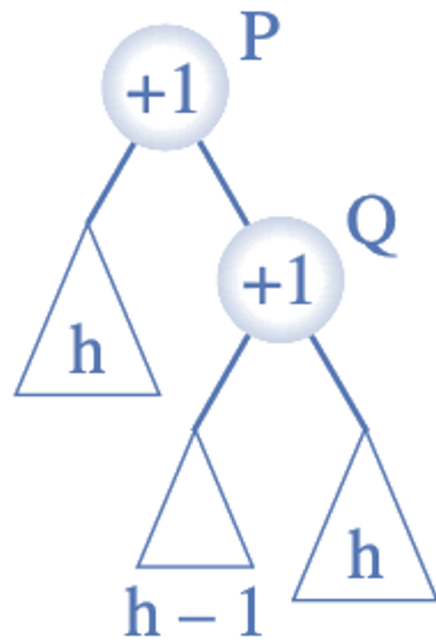
(c)

c) Cân bằng cây

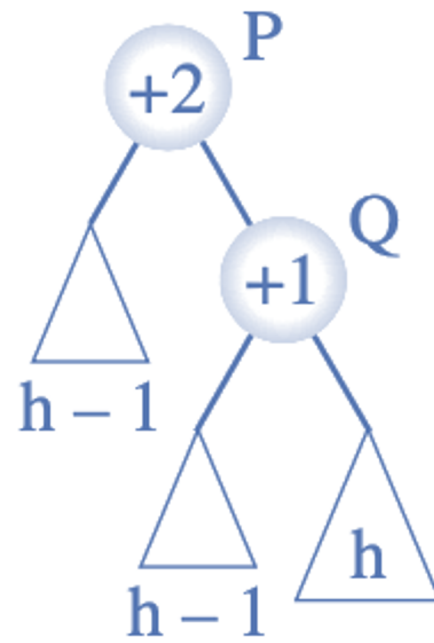
IV - Balancing a Tree - AVL Trees - Node Deletion

- Xóa nút trong cây AVL yêu cầu việc cập nhật các nhân tố cân bằng của các nút con của nút cha của nút bị xóa.
- Các nút có nhân tố cân bằng ± 2 cần được cân bằng lại

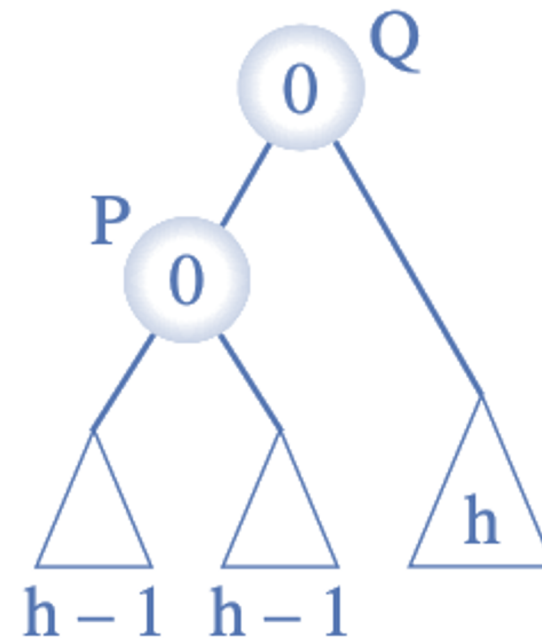
IV - Balancing a Tree - AVL Trees - Node Deletion



(a)

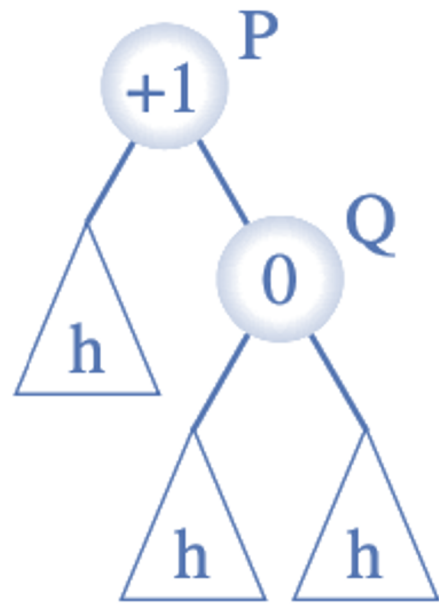


(b)

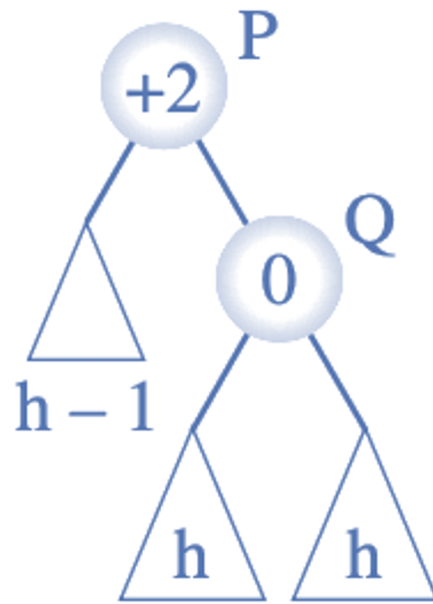


(c)

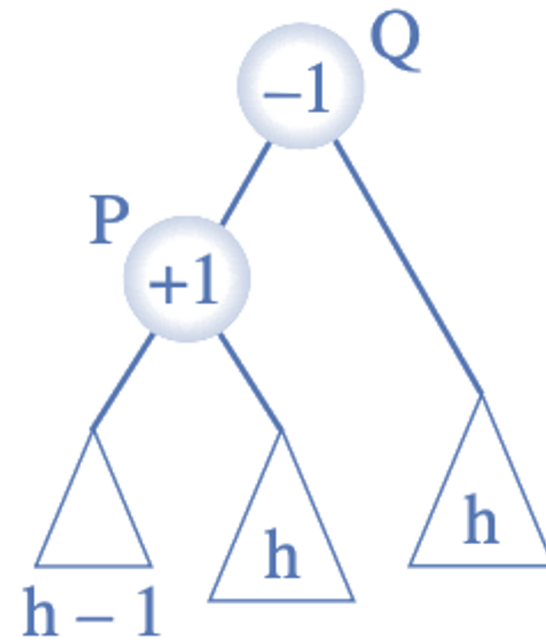
IV - Balancing a Tree - AVL Trees - Node Deletion



(d)

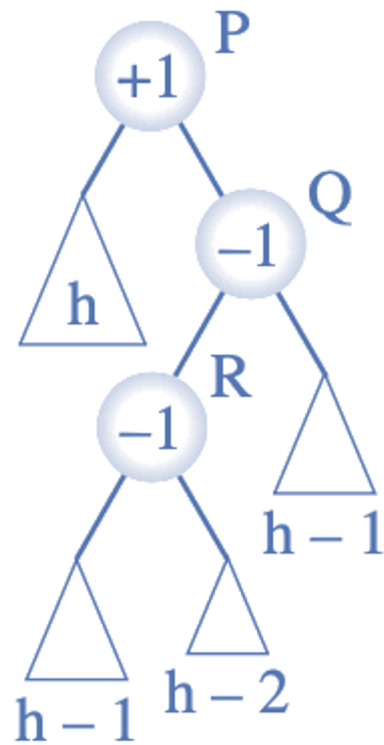


(e)

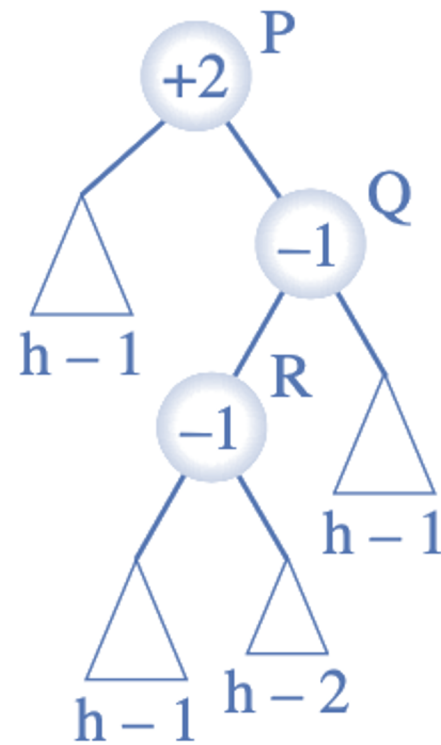


(f)

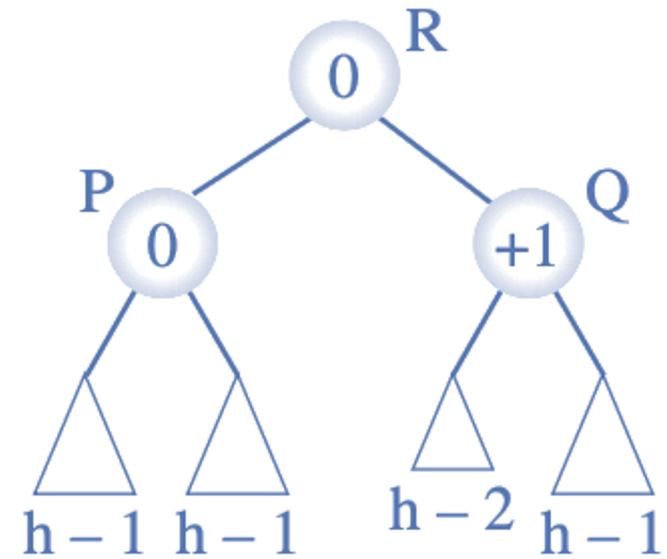
IV - Balancing a Tree - AVL Trees - Node Deletion



(g)

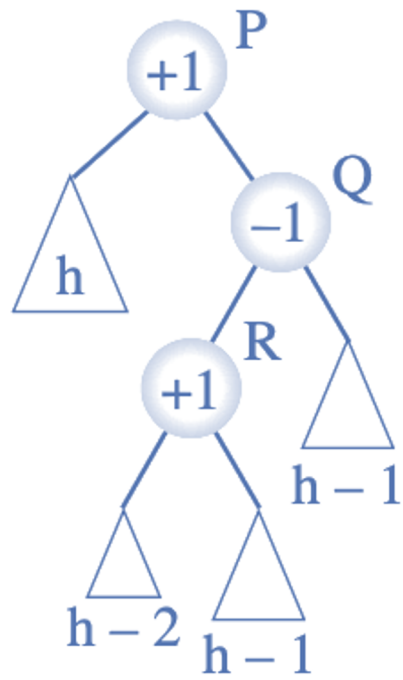


(h)

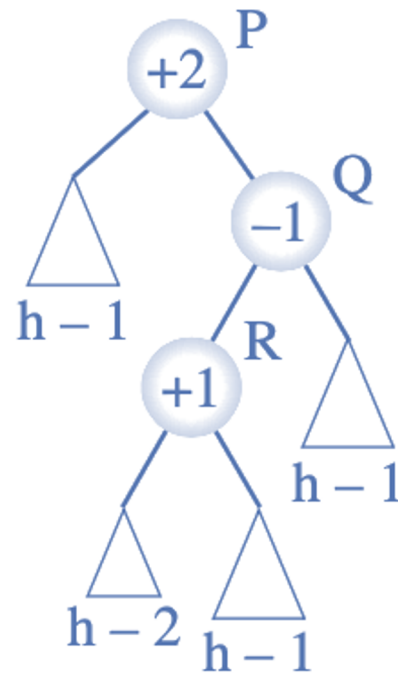


(i)

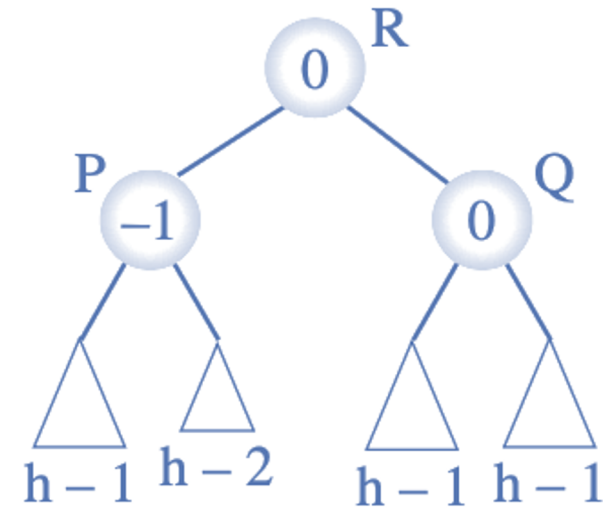
IV - Balancing a Tree - AVL Trees - Node Deletion



(j)



(k)



(l)

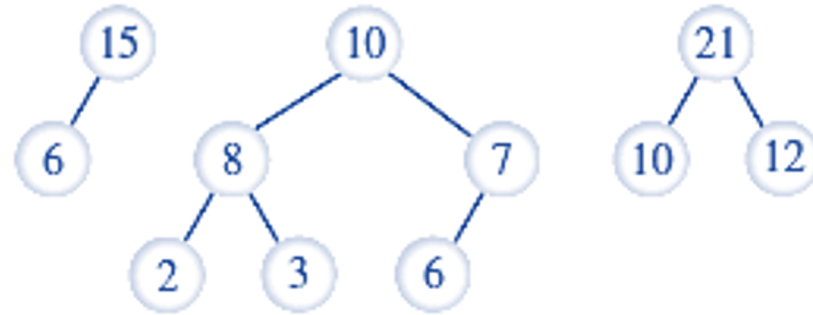
Max heap là một cấu trúc đặc biệt của cây nhị phân với các đặc trưng sau

- Giá trị của nút gốc **lớn hơn hoặc bằng** giá trị của các nút con của nó
- Cây được cân bằng hoàn hảo và các nút lá tại mức cao nhất ở phía bên trái của cây.

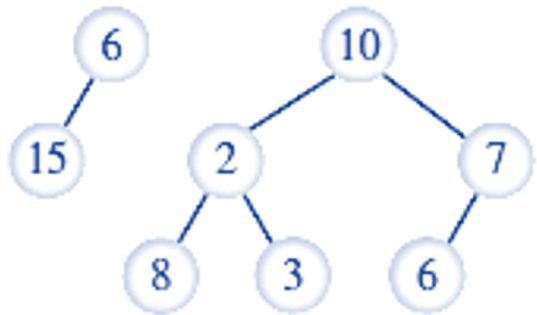
Min heap là một cấu trúc đặc biệt của cây nhị phân với các đặc trưng sau

- Giá trị của nút gốc **nhỏ hơn hoặc bằng** giá trị của các nút con của nó
- Cây được cân bằng hoàn hảo và các nút lá tại mức cao nhất ở phía bên trái của cây.

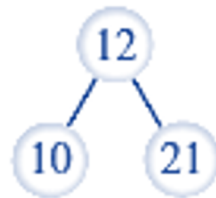
V - Heaps



(a)



(b)



(c)

a) Heap

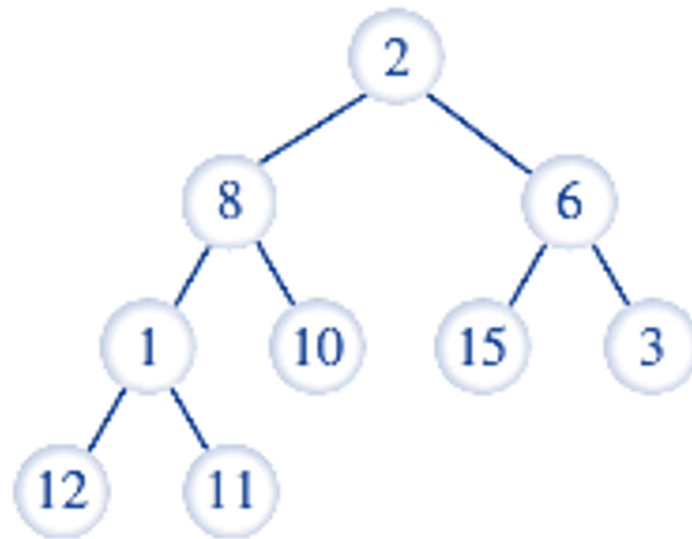
b) Không phải heap vì không thỏa điều kiện đầu

c) Không phải heap vì không thỏa điều kiện sau

V - Heaps

Heap có thể được biểu diễn bằng mảng theo thứ tự từ trên xuống dưới từ trái qua phải:

data=[2 8 6 1 10 15 3 12 11]



V - Heaps

Heap có thể được định nghĩa bởi mảng heap với độ dài n

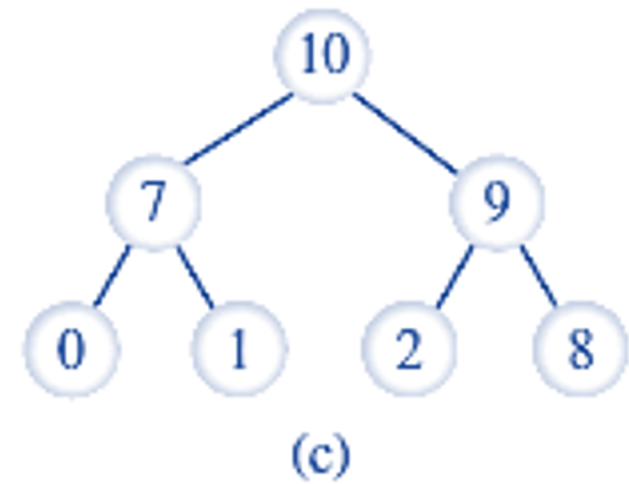
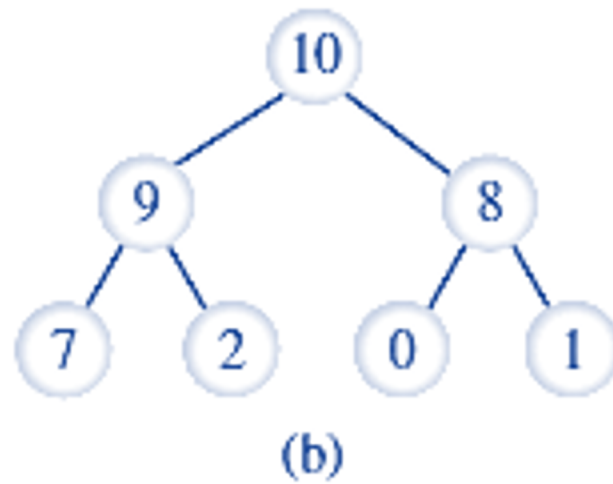
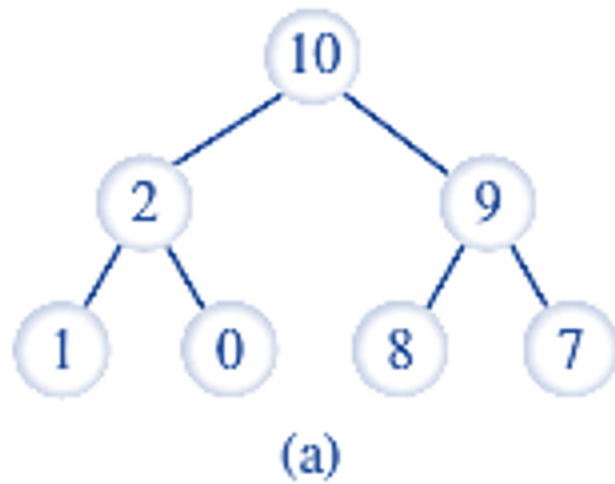
$$\text{heap}[i] > \text{heap}[2i + 1], \text{ for } 0 < i < (n - 1) / 2$$

và

$$\text{heap}[i] > \text{heap}[2i + 2], \text{ for } 0 < i < (n - 2) / 2$$

- Phần tử trong max heap không được sắp xếp theo thứ tự. Chỉ đảm bảo nút gốc là phần tử lớn nhất và các nút còn lại nhỏ hơn hoặc bằng nút gốc.

V - Heaps



b) heap có thứ tự tốt nhất

V - Heaps

- Heap biểu diễn hàng đợi ưu tiên (priority queue)
- Heap hiệu quả hơn DSLK để biểu diễn hàng đợi ưu tiên.
- Heap là cây cân bằng hoàn hảo, tìm kiếm trên cấu trúc này đạt hiệu quả cao.
- Nút mới sẽ được thêm vào (enqueue) nút lá. Nếu cây cần phải được cấu trúc lại, nút mới thêm sẽ được di chuyển từ nút lá về phía nút gốc.
- Nút được xóa (dequeue) ra khỏi hàng đợi ưu tiên sẽ là nút gốc. Các nút khác sẽ được di chuyển xuống nút lá tới vị trí phù hợp.

V - Heaps

`heapEnqueue(e1)`

put e1 at the end of the heap;

while e1 is not in the root and $e1 > \text{parent}(e1)$

swap e1 with its parent;

`heapDequeue()`

extract the element from the root;

put the element from the last leaf in its place;

remove the last leaf;

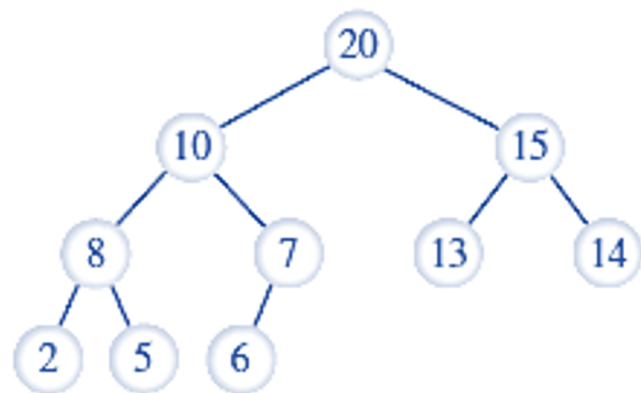
// both subtrees of the root are heaps

p = the root;

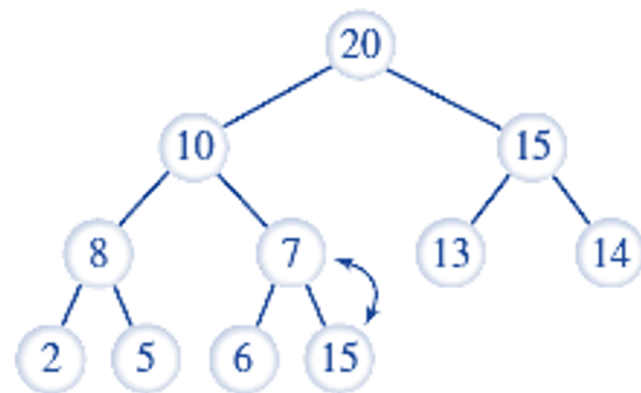
while p is not a leaf and $p < \text{any of its children}$

swap p with the larger child;

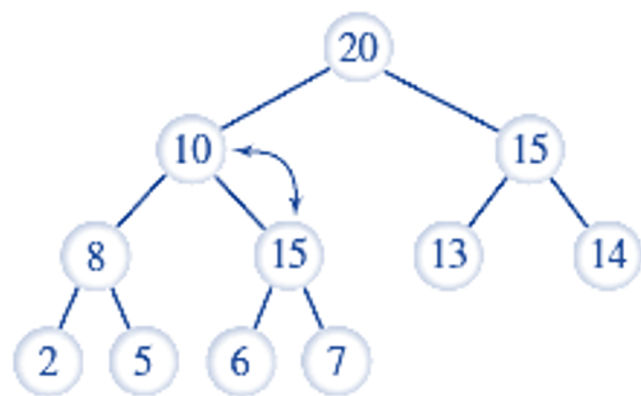
V - Heaps



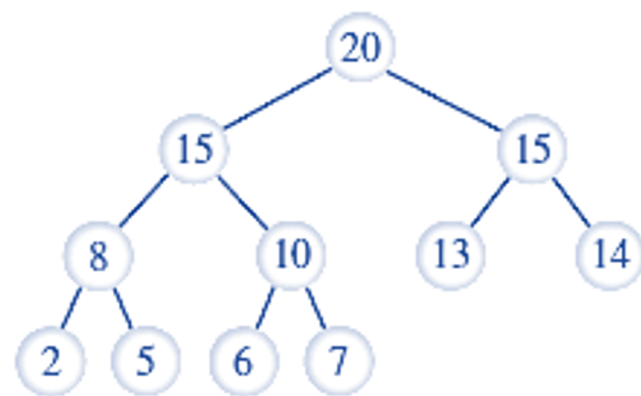
(a)



(b)

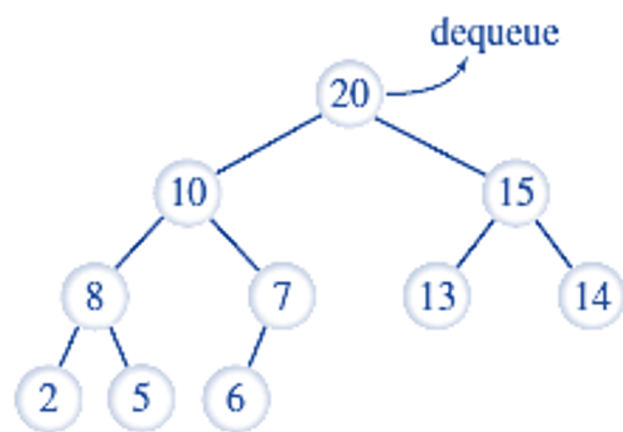


(c)

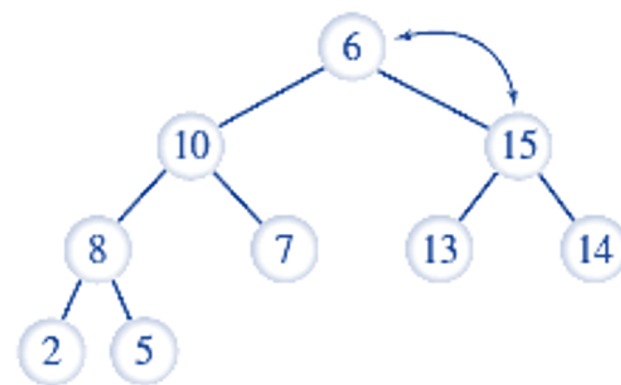


(d)

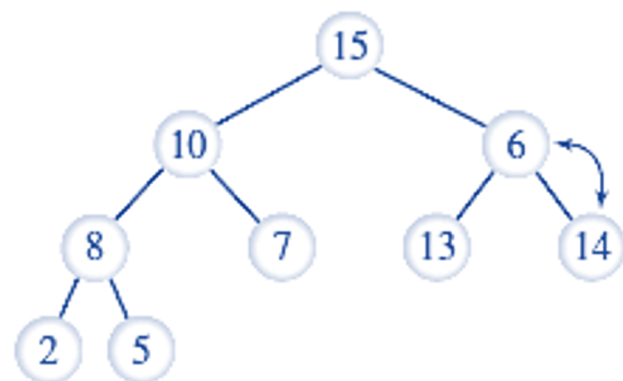
V - Heaps



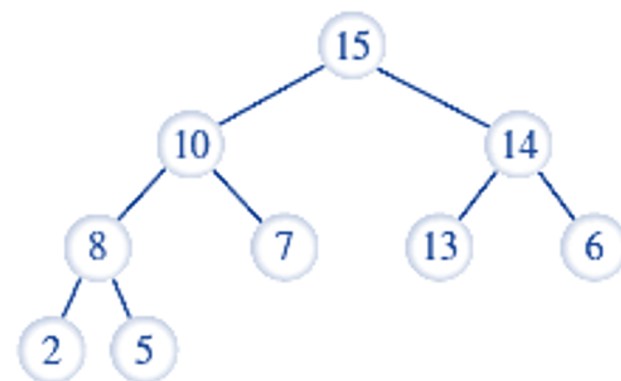
(a)



(b)



(c)



(d)

1. Viết định nghĩa hàm để xác định một cây nhị phân là cây tìm kiếm nhị phân
2. Viết định nghĩa hàm để xác định một cây nhị phân là cây nhị phân cân bằng hoàn hảo

THANK YOU FOR YOUR ATTENTIONS